Transformers
&
Intro to LLMs

Michalis Vazirgiannis

Distinguished Prof. @Ecole Polytechnique, IPP, France

http://www.lix.polytechnique.fr/dascim

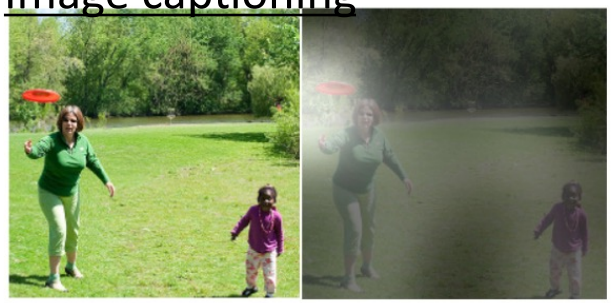
October 2025

Outline

- **Transformer,**
- Pretraining LLMs
- LLMs Instruction
- Prompting LLMs
- Parameter Efficient Fine Tuning

Attention is ubiquitous in DL today

Image captioning



A woman is throwing a frisbee in a park.



A dog is standing on a hardwood floor.



A stop sign is on a road with a mountain in the background.



A little girl sitting on a bed with a teddy bear.



A group of people sitting on a boat in the water.



A giraffe standing in a forest with trees in the background.

Show, attend and tell: Neural image caption generation with visual attention (Xu et al. 2015)

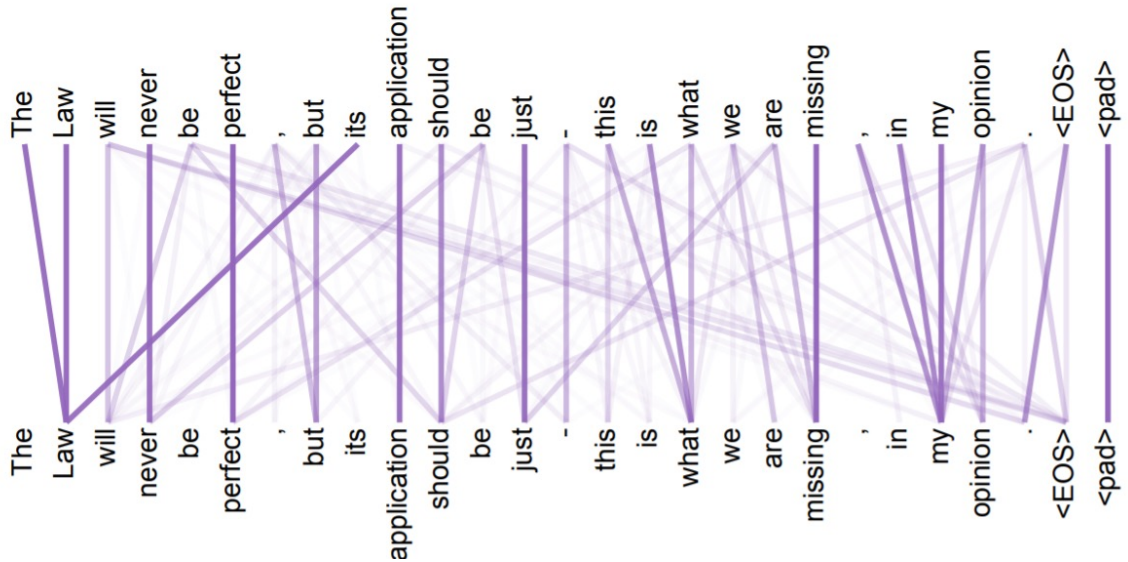
Attention is All You Need - Transformer

TITLE	CITED BY	YEAR
Attention is all you need A Vaswani, N Shazeer, N Parmar, J Uszkoreit, L Jones, AN Gomez, ... Advances in neural information processing systems 30	209415	2017

Attention 👀

"The computer which I had just put into the machine room on the fifth floor crashed."

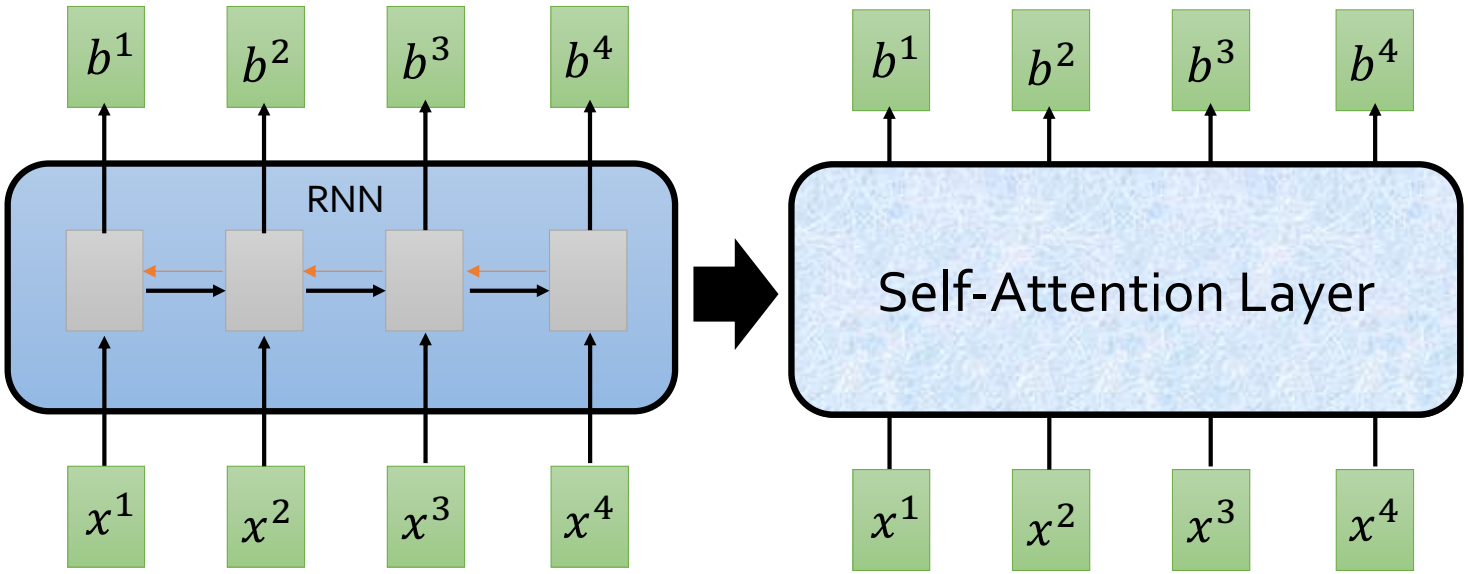
- Core idea: build a mechanism to focus ("attend") on a particular part of the context.
- How can this overcome the "long-range dependencies" problem in RNNs? By allow the model to **directly "look"** 👀 at all tokens and decide which one is useful.



[Attention Is All You Need, Vaswani et al. 2017]

Self-Attention

- b^i is obtained based on the whole input sequence.
- can be parallelly computed.

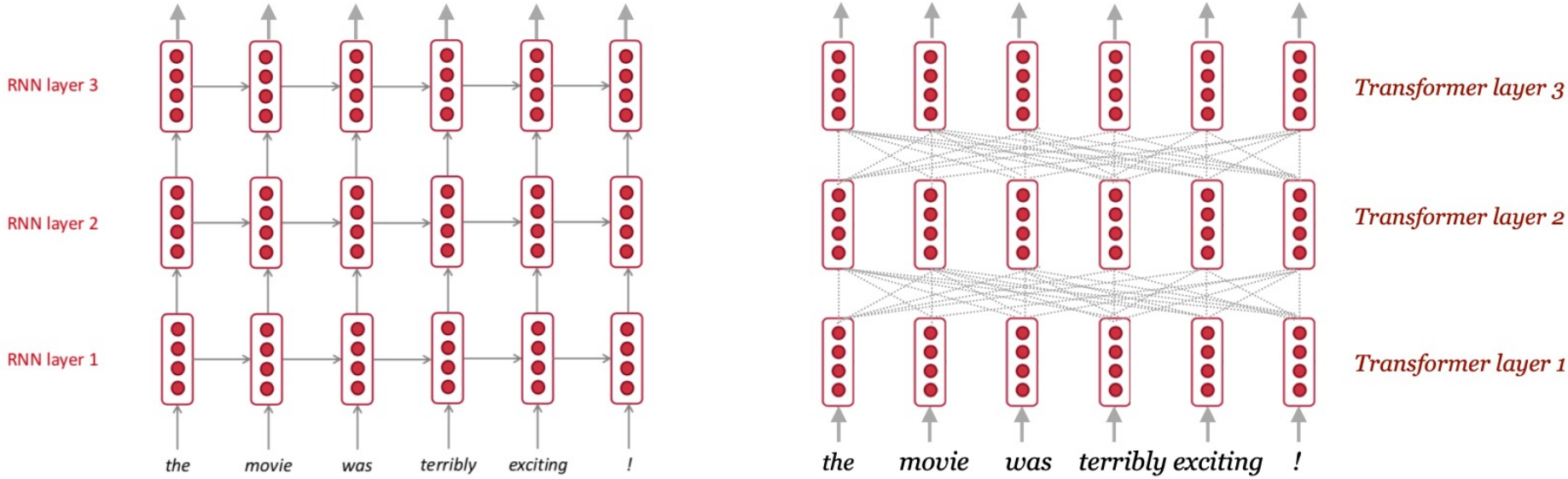


Idea: replace any thing done by RNN with self-attention.

"Neural machine translation by jointly learning to align and translate" Bahdanau etl. 2014;
"Attention is All You Need" Vaswani et al. 2017

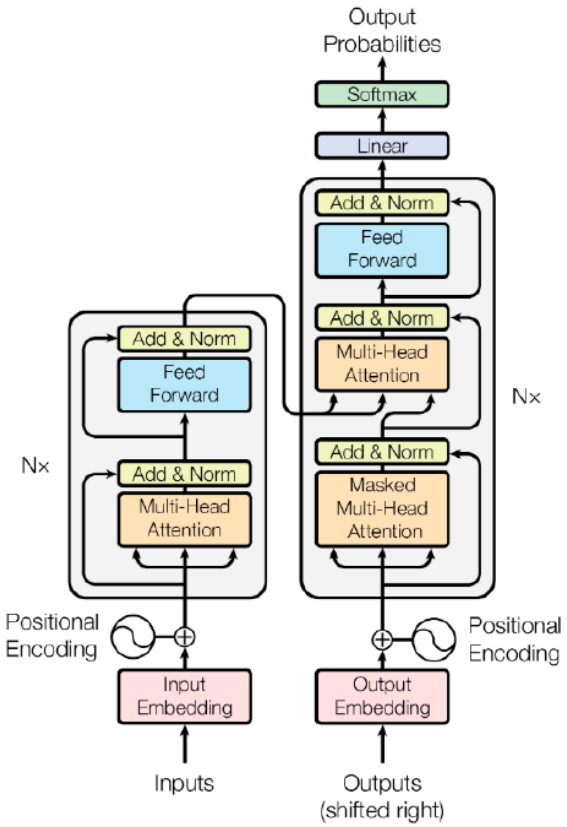
RNN vs Transformer

- Notice that self-attention can directly "look" at all tokens and decide which one is useful.



Transformer

- A model that follows the encoder-decoder structure, where the input tokens are mapped to a sequence of continuous representations, and these representations are consumed by the decoder which generates a sequence of outputs.
- Does not use recurrent neural networks or convolutional neural networks. Instead, it only uses dense and attention layers.



The weather is nice <cls> Le temp est beau

Defining Self-Attention

- Terminology:
  - Query: to match others
  - Key: to be matched
  - Value: information to be extracted

[Vaswani et al. 2017: https://arxiv.org/abs/1706.03762]

q: query (to match others)

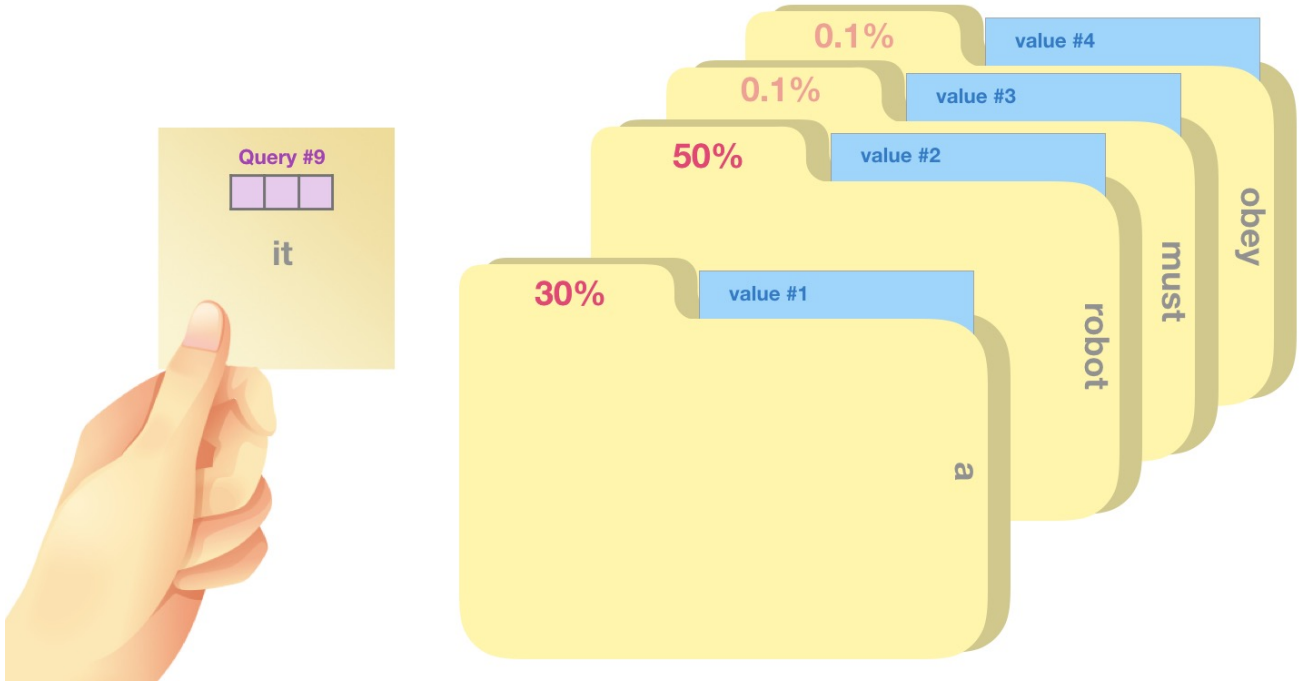
$$q_i = x_i W^q$$

k: key (to be matched)

$$k_i = x_i W^k$$

v: value (information to be extracted)

$$v_i = x_i W^v$$



q: query (to match others)

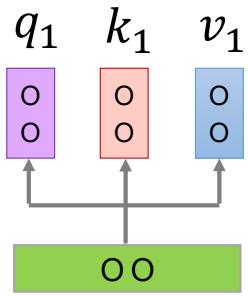
$$q_i = x_i W^q$$

k: key (to be matched)

$$k_i = x_i W^k$$

v: value (information to be extracted)

$$v_i = x_i W^v$$



The

q: query (to match others)

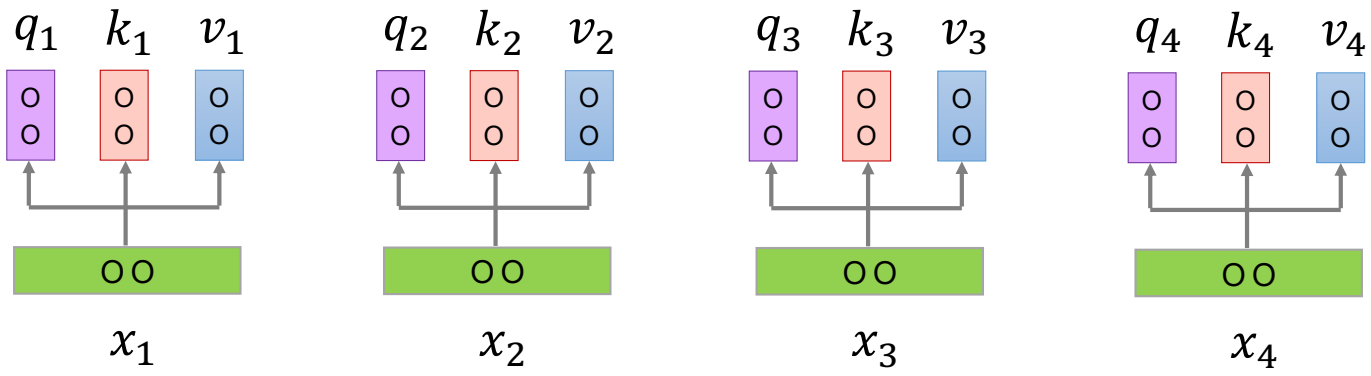
$$q_i = x_i W^q$$

k: key (to be matched)

$$k_i = x_i W^k$$

v: value (information to be extracted)

$$v_i = x_i W^v$$



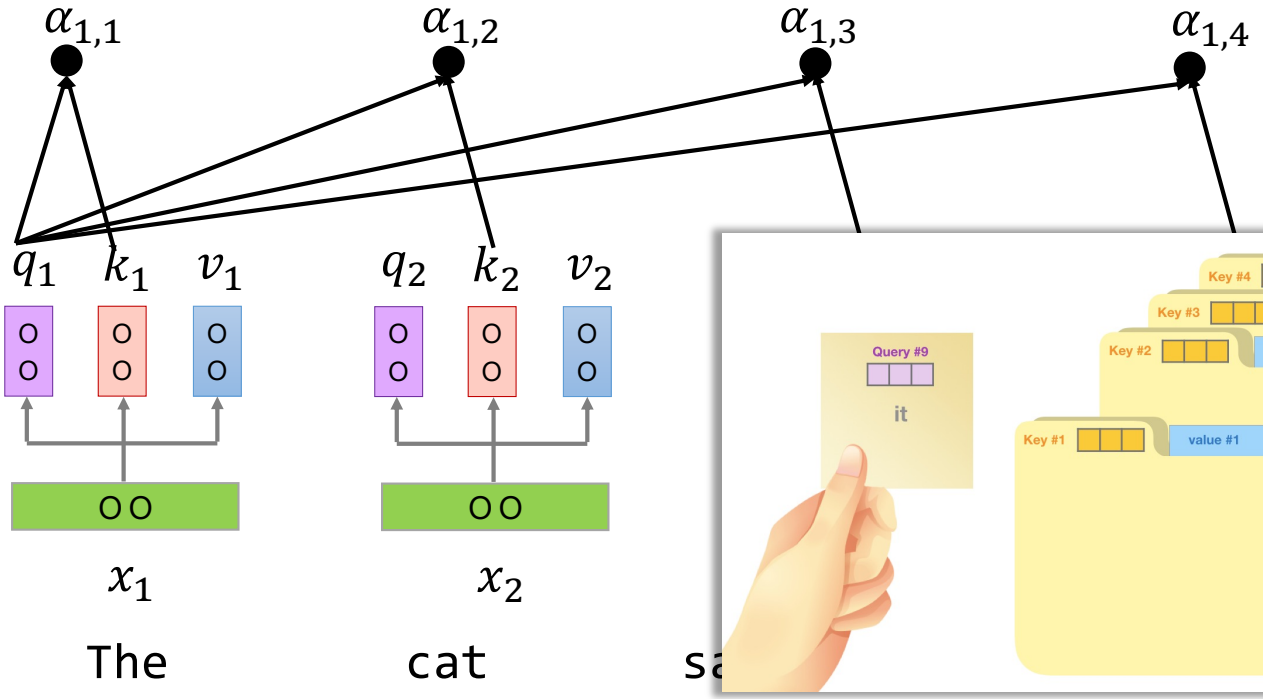
$$\alpha_{1,i} = \underbrace{q^1 \cdot k^i / \sqrt{d}}_{\text{Scaled dot product}}$$

q: query (to match others)

k: key (to be matched)

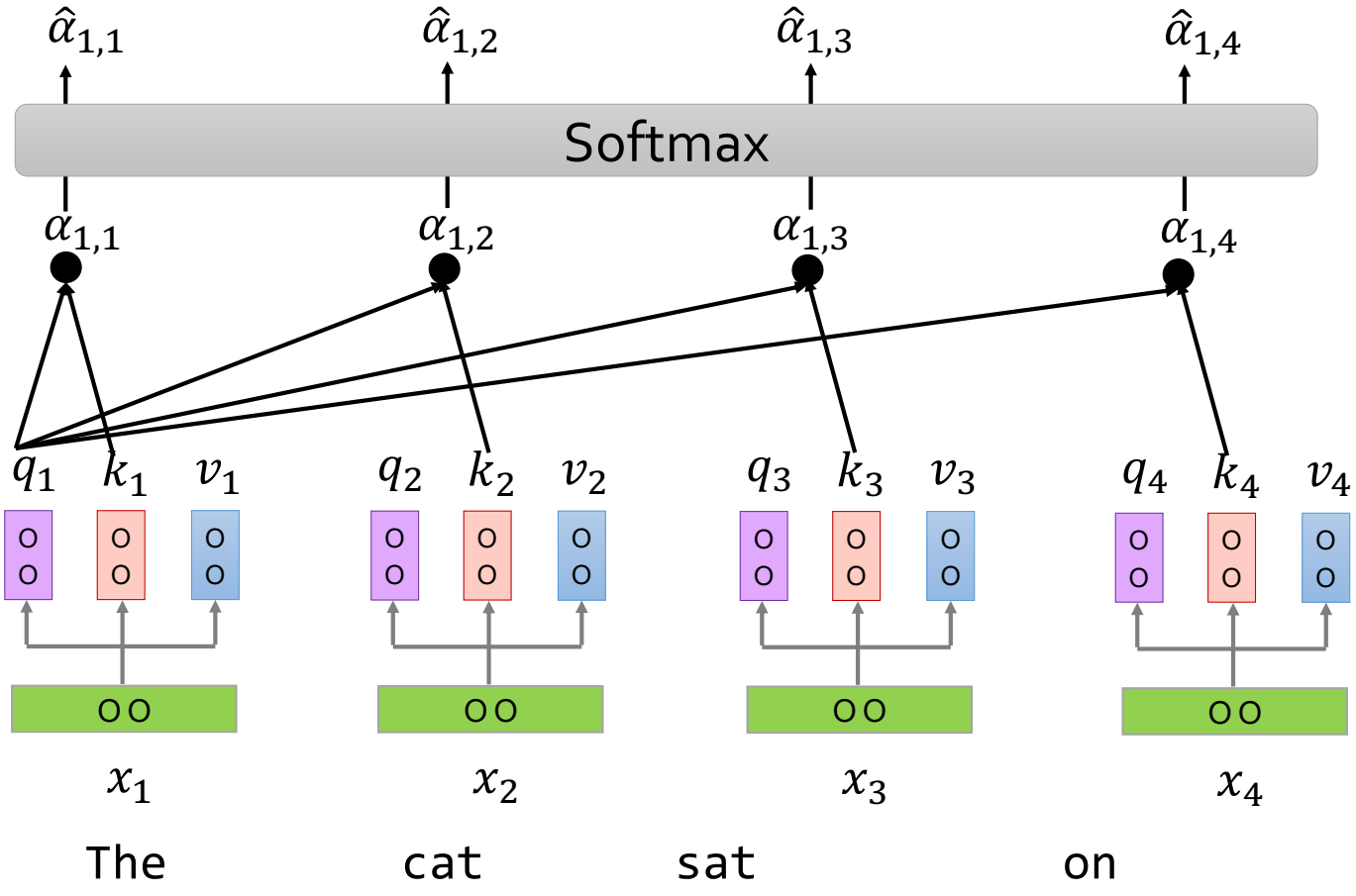
v: value (information to be extracted)

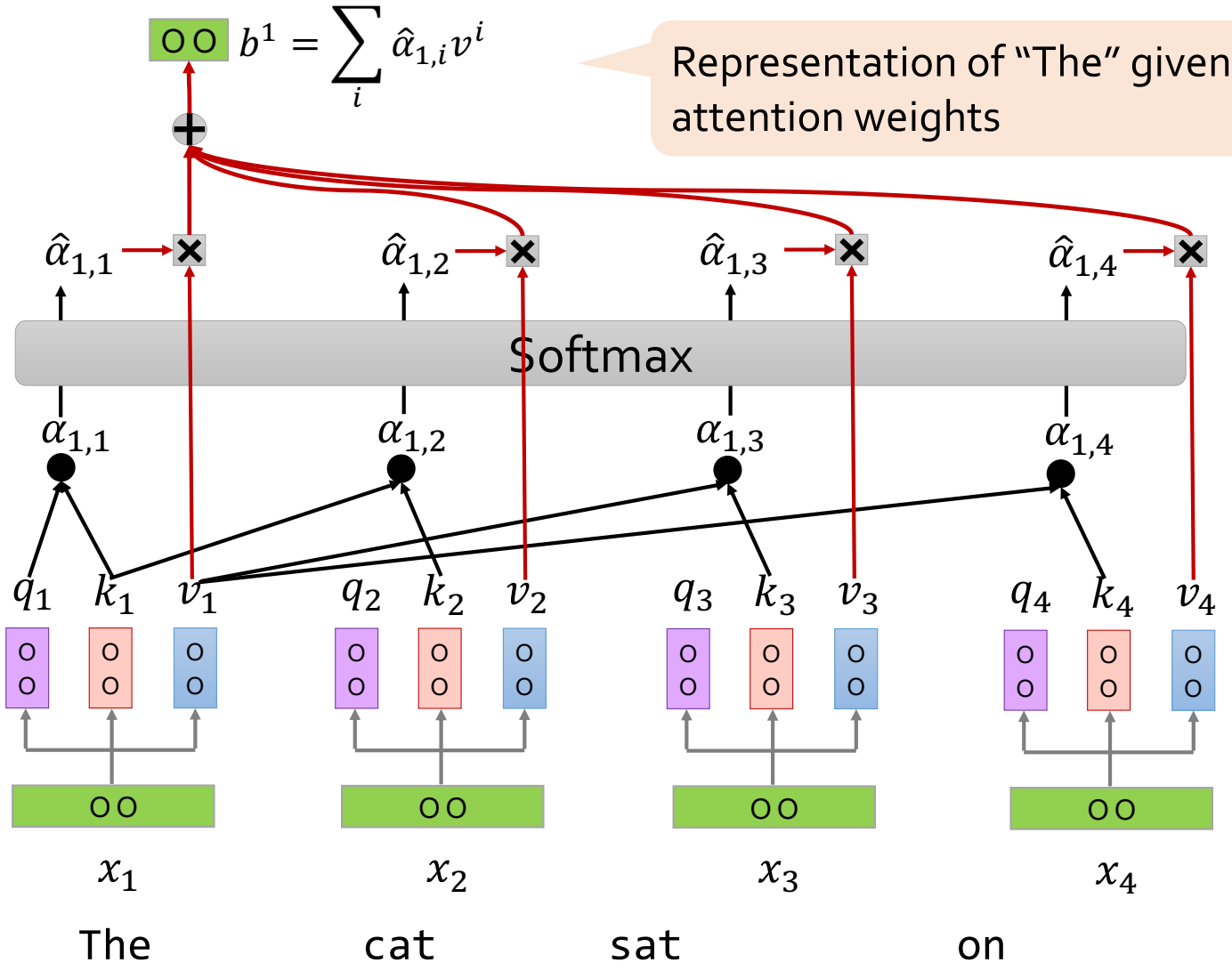
How much should "The" attend to other positions?



$$\sigma(z)_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

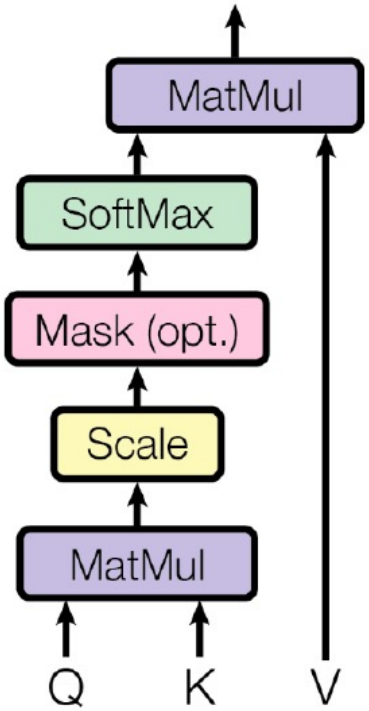
How much should "The" attend to other positions?




$b^1 = \sum_i \hat{\alpha}_{1,i} v^i$
Representation of "The" given the attention weights
$\hat{\alpha}_{1,1}$
$\hat{\alpha}_{1,2}$
$\hat{\alpha}_{1,3}$
$\hat{\alpha}_{1,4}$
Softmax
$\alpha_{1,1}$
$\alpha_{1,2}$
$\alpha_{1,3}$
$\alpha_{1,4}$
q_1 k_1 v_1
q_2 k_2 v_2
q_3 k_3 v_3
q_4 k_4 v_4
x_1
x_2
x_3
x_4
The
cat
sat
on

Transformer – Scaled dot product attention

- A query Q_i representing the token at position i attend to a sequence of tokens (represented by K): $Q_i K^T$.
- The result scaled by $\sqrt{d_k}$ (dimension of Q_i) is passed to a softmax function to compute a score for each of the tokens: $softmax(\frac{Q_i K^T}{\sqrt{d_k}})$
- A weighted sum of the Value vectors is calculated as the new representation of the token at position i: $softmax(\frac{Q_i K^T}{\sqrt{d_k}})V$
- Queries are stacked in one matrix Q, the output of the attention layer becomes: $softmax(\frac{QK^T}{\sqrt{d_k}})V$



Query - Key - Value

computations of vector representation of tokens “thinking” and “machines” IN CONTEXT via attention.

- W^Q, W^K, W^V are trainable matrices
- q,k,v vectors corresponding to the tokens in the context.
- For “thinking”: q1*k1 is the self similarity for this word which is normalized and softmaxed. The result multiplied by v_1.

- z1 = sum (q1, k1, v1) that represent “thinking” in the context of “thinking”).
- Similar process happens among “Thinining” and “Machines” resulting in the z2 vector that represents after all computations the vector that corresponds to “thinking” in the context of “machines”
- The W^* matrices represent attention weights among the tokens in the context.. ?

Zij representation of word i to the head j – then we MERGE and give them to the next layer.

- K, W^iV are intialised randomly and are trainable – i stand for the head
- X1 : [“Thinking” embedding: positional embedding of “Thinking”]
- q1 = X1 * W^Q, k1 = X1 * W^K, v1 = X1 * W^V
- qi*kj : attention among tokens i and j. Then normalized by softmax
- z1 = sum(v1j): representation of token 1 in context, j: words in context
- zi ‘s concatenated and passed to the next attention layers…

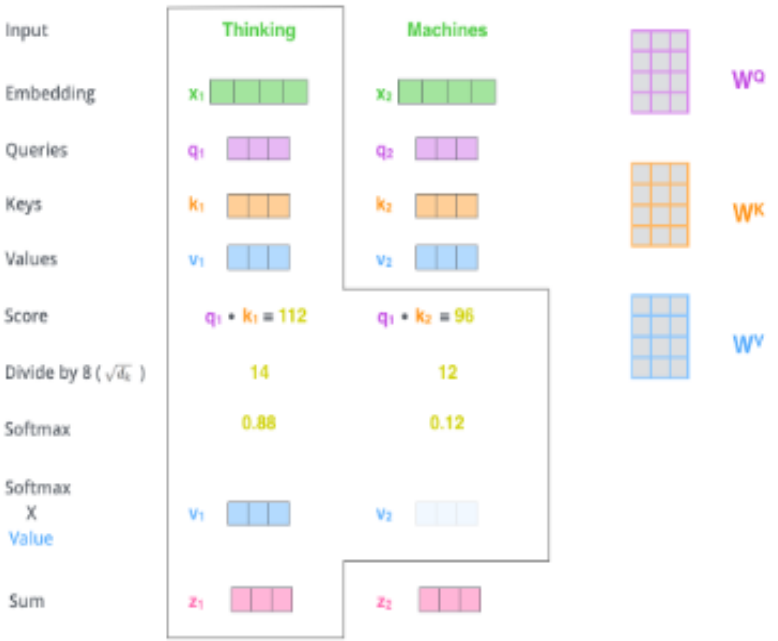
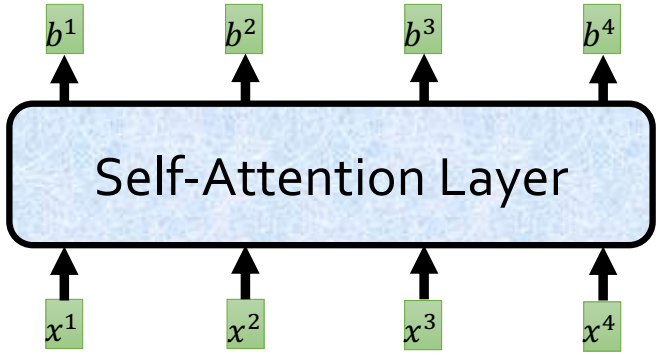


Figure: Self-attention calculation.

- Multi-Head Attention: $\{W_0^Q, W_0^K, W_0^V\}, \{W_1^Q, W_1^K, W_1^V\}, \ldots$

Recap: Self-Attention

- Attention is a powerful mechanism to create context-aware representations
- A way to focus on select parts of the input

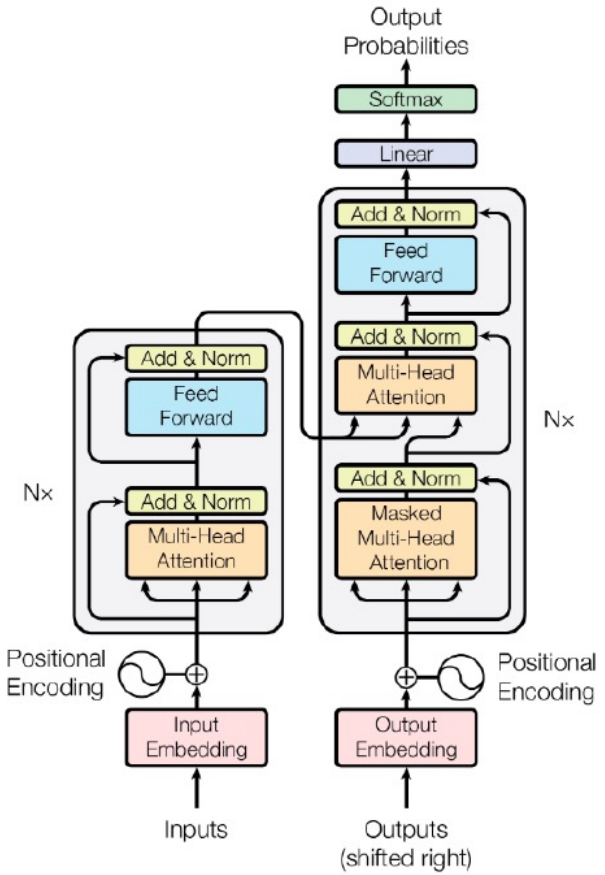


- Better at maintaining long-distance dependencies in the context.

[Attention Is All You Need, Vaswani et al. 2017]

Transformers- Attention

- **Self-attention:** Applied in the encoder and in the decoder:
  - In the encoder: Q, K and V come from the previous encoder sub-layer.
  - In the decoder: Q, K and V come from the previous decoder layer and a mask is applied to prevent the decoder from attending future positions.
- **Encoder-decoder attention:** Applied only in the decoder. Q come from the previous decoder layer. K and V come from the output representations of the encoder.



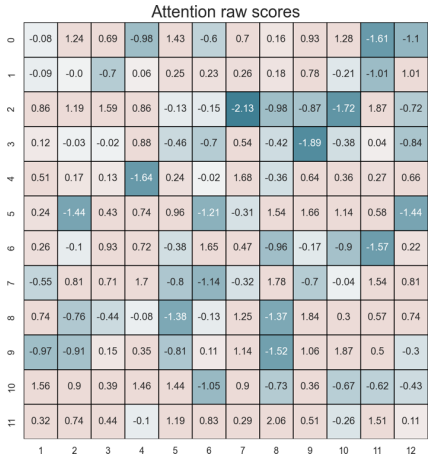
Self-Attention: Matrix Notation

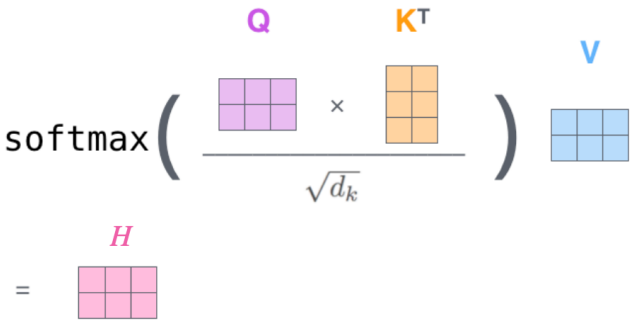
- Input sequence: $\mathbf{x} = [x_1, x_2, \ldots, x_n] \in \mathbb{R}^{n \times d}$
- Calculate:

$$\boldsymbol{Q} = \mathbf{x}\mathbf{W}^q \in \mathbb{R}^{n \times d}$$
$$\boldsymbol{K} = \mathbf{x}\mathbf{W}^k \in \mathbb{R}^{n \times d}$$
$$\boldsymbol{V} = \mathbf{x}\mathbf{W}^v \in \mathbb{R}^{n \times d}$$

- Pairwise similarity matrix between queries and keys: $\boldsymbol{Q}\boldsymbol{K}^T \in \mathbb{R}^{n \times n}$
- Attention output:

$$\text{Attention}(\mathbf{x}) = \text{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^{\mathrm{T}}}{\sqrt{d}}\right)\boldsymbol{V} \in \mathbb{R}^{n \times d}$$

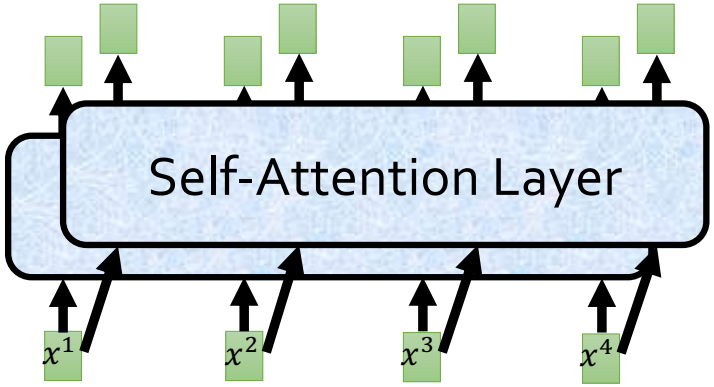




Multi-Headed Self-Attention



- **Issue:** Each head has limited capacity, e.g., it cannot attend to too many things at the same time.
- **Main idea:** Allows model to jointly attend to information from different representation subspaces (like ensembling).
- Multiple parallel attention layers.
  - Each attention layer has its own parameters.
  - Concatenate the results and run them through a linear projection.



[Attention Is All You Need, Vaswani et al. 2017]

Multi-Headed Self-Attention

- Just concatenate all the heads and apply an output projection.

$$\text{head}_i = \text{Attention}\left(\mathbf{x}\mathbf{W}_i^q, \mathbf{x}\mathbf{W}_i^k, \mathbf{x}\mathbf{W}_i^v\right)$$
$$\text{MultiHeadedAttention}(\mathbf{x}) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\boldsymbol{W}^O$$

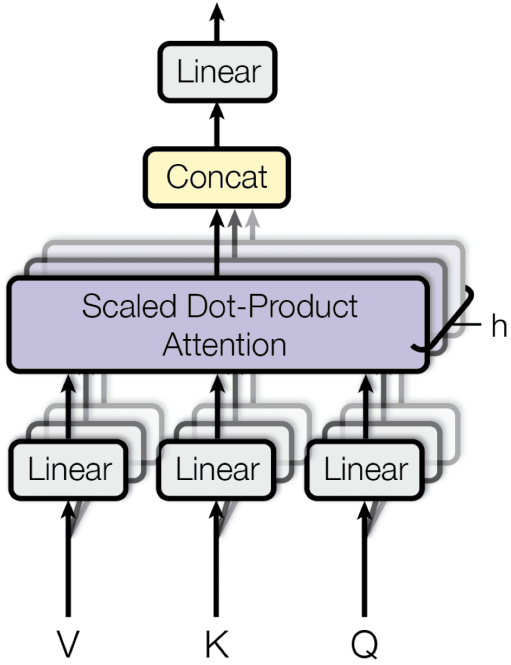
- Previously, used the following dimensions for **single**-head SA:

$$\mathbf{W}_i^q \in \mathbb{R}^{d\times d}, \quad \mathbf{W}_i^k \in \mathbb{R}^{d\times d}, \quad \mathbf{W}_i^v \in \mathbb{R}^{d\times d},$$

- In practice, use a reduced dimension for each head.

$$\mathbf{W}_i^q \in \mathbb{R}^{d\times \frac{d}{h}}, \quad \mathbf{W}_i^k \in \mathbb{R}^{d\times \frac{d}{h}}, \quad \mathbf{W}_i^v \in \mathbb{R}^{d\times \frac{d}{h}}, \quad \boldsymbol{W}^O \in \mathbb{R}^{d\times d}$$

- The total computational cost is similar to that of single-head attention with full dimensionality.



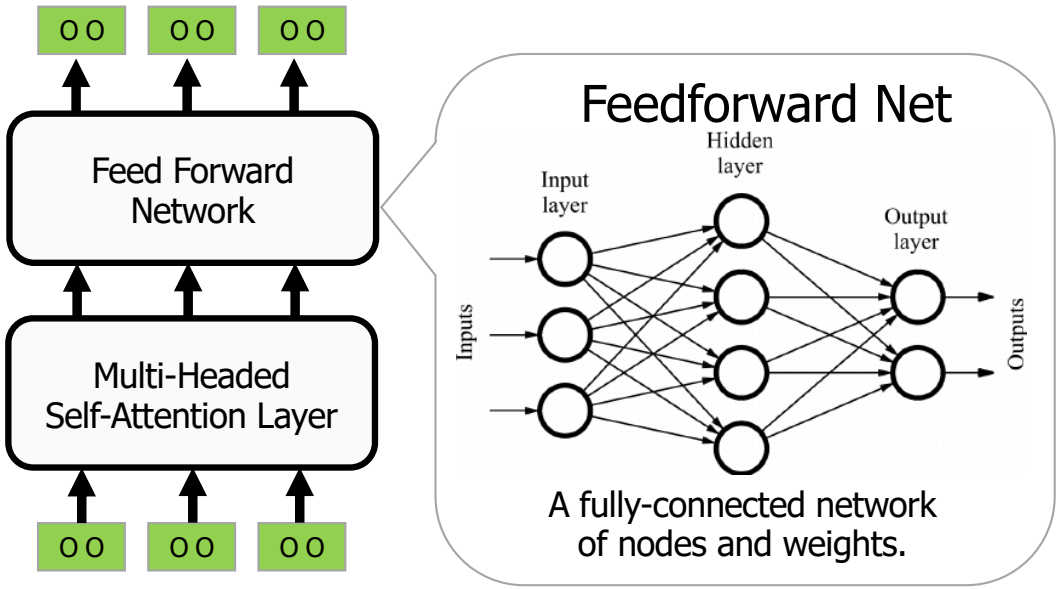
h: number of heads
d: feature dimension
in output of SA

Combine with FFN

- Add a feed-forward network to add more expressivity.
  - This applies another nonlinearity to the representations (or "post-process" them).

$$\text{FFN}(\mathbf{x}) = f(\mathbf{x}\boldsymbol{W}_1 + b_1)\boldsymbol{W}_2 + \boldsymbol{b}_2$$
$$\boldsymbol{W}_1 \in \mathbb{R}^{d \times d_{\text{ff}}},$$
$$\boldsymbol{W}_2 \in \mathbb{R}^{d_{\text{ff}} \times d}$$

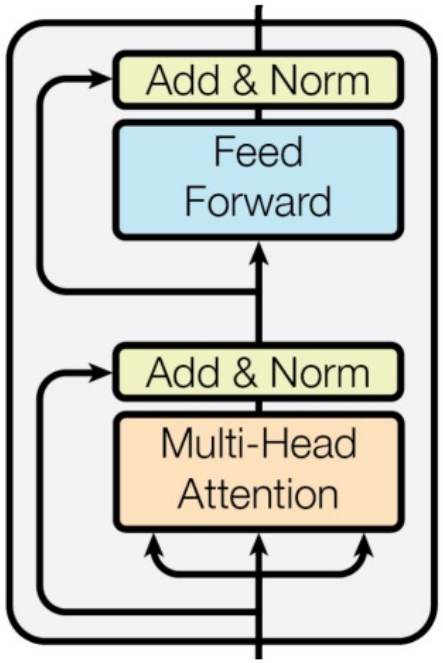
  - Usually, the dimensionality of the hidden feedforward layer d_{ff} is 2-8 times larger than the input dimension d.



How Do We Prevent Vanishing Gradients?

- Residual connections let the model "skip" layers
  - These connections are particularly useful for training deep networks

- Use layer normalization to stabilize the network and allow for proper gradient flow



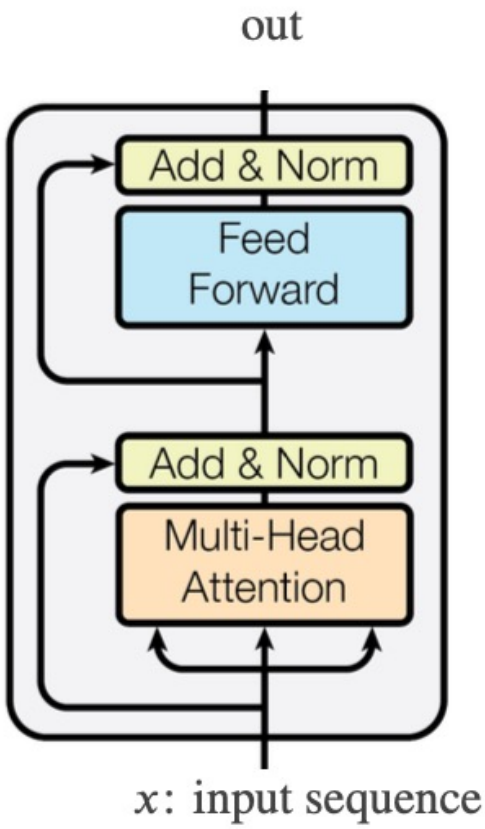
[Attention Is All You Need, Vaswani et al. 2017]

Putting it Together: Self-Attention Block

Given input $\mathbf{x}$:

$$\tilde{\mathbf{x}} = \text{MultiHeadedAttention}(\mathbf{x}; \mathbf{W}^q, \mathbf{W}^k, \mathbf{W}^v)$$
$$\mathbf{x} = LayerNorm\ (\tilde{\mathbf{x}} + \boldsymbol{x})$$

$$\tilde{\mathbf{x}} = \text{FFN}(\mathbf{x}) = f(\mathbf{x}W_1 + b_1)W_2 + b_2$$
$$\text{out} = LayerNorm(\tilde{\mathbf{x}} + \mathbf{x})$$

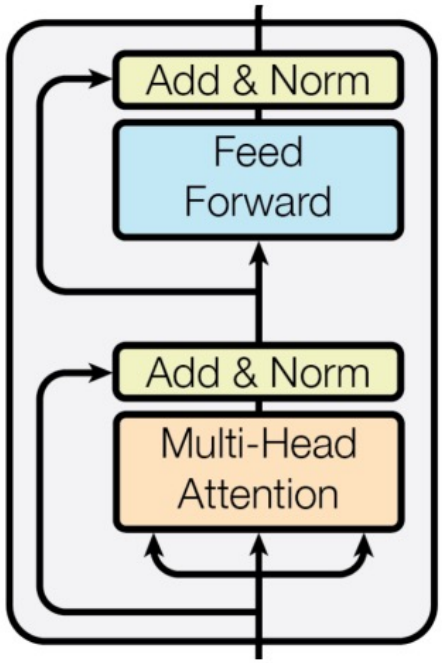


[Attention Is All You Need, Vaswani et al. 2017]

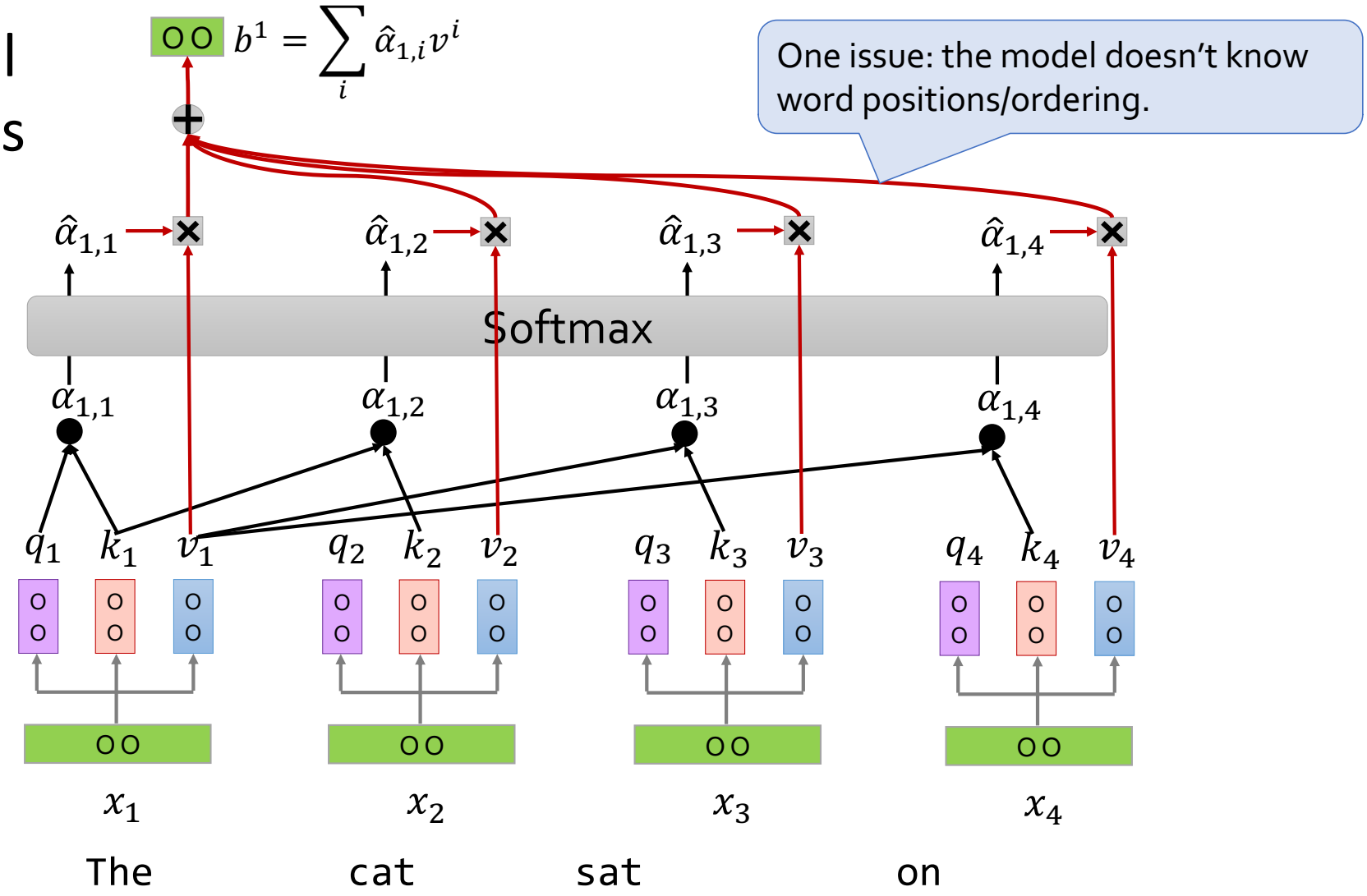
Summary: Self-Attention Block

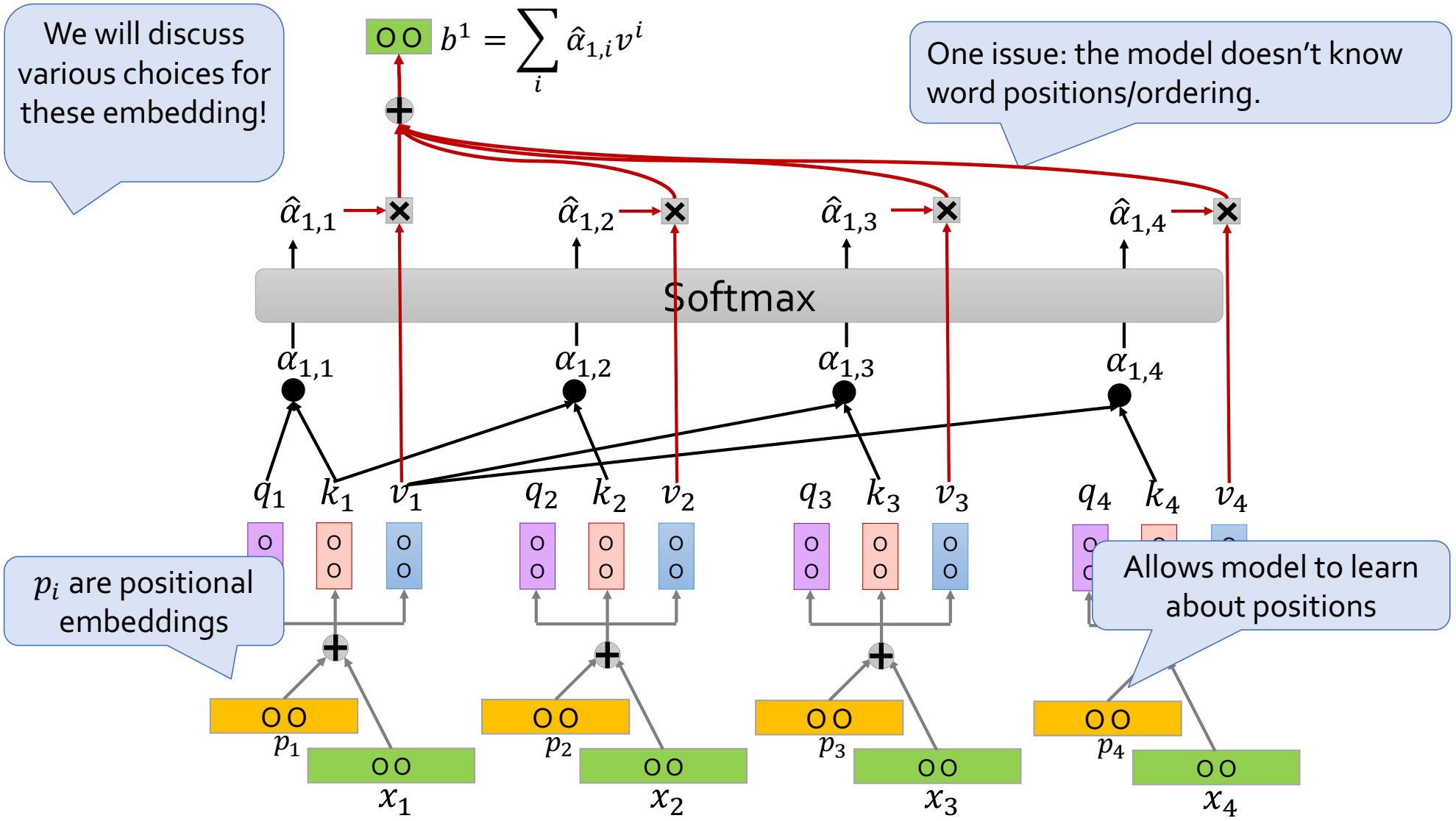
- **Self-Attention:** A critical building block of modern language models.
  - The idea is to compose meanings of words weighted according some similarity notion.

- **Next:** We will combine self-attention blocks to build various architectures known as Transformer.



Positional encodings




We will discuss various choices for these embedding!
$b^1 = \sum_i \hat{\alpha}_{1,i} v^i$
One issue: the model doesn't know word positions/ordering.
$\hat{\alpha}_{1,1}$
$\hat{\alpha}_{1,2}$
$\hat{\alpha}_{1,3}$
$\hat{\alpha}_{1,4}$
Softmax
$\alpha_{1,1}$
$\alpha_{1,2}$
$\alpha_{1,3}$
$\alpha_{1,4}$
q_1 k_1 v_1
q_2 k_2 v_2
q_3 k_3 v_3
q_4 k_4 v_4
p_i are positional embeddings
Allows model to learn about positions
p_1
p_2
p_3
p_4
x_1
x_2
x_3
x_4

Absolute Positional Embeddings

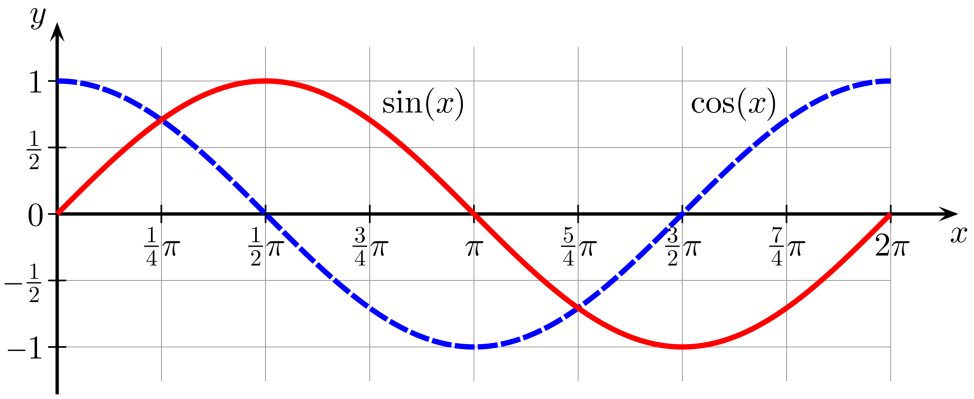
- The idea is to create vectors that uniquely encoder each position.
- For example, consider vectors of binary values.
  - Example below shows 4-dimensional position encodings for 16 positions.

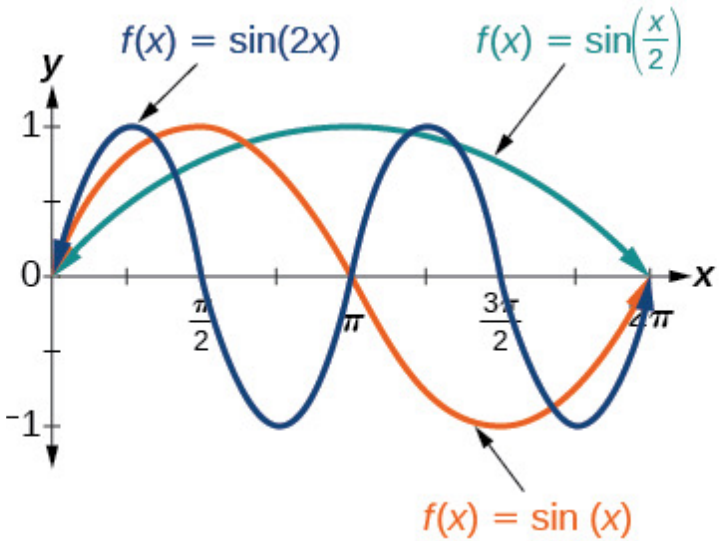
0 :	0	0	0	0	8 :	1	0	0	0
1 :	0	0	0	1	9 :	1	0	0	1
2 :	0	0	1	0	10 :	1	0	1	0
3 :	0	0	1	1	11 :	1	0	1	1
4 :	0	1	0	0	12 :	1	1	0	0
5 :	0	1	0	1	13 :	1	1	0	1
6 :	0	1	1	0	14 :	1	1	1	0
7 :	0	1	1	1	15 :	1	1	1	1

Issues: Discrete numbers; the information is localized around a few bits; embedding depends on the number of positions ☹

https://kazemnejad.com/blog/transformer_architecture_positional_encoding/

Math Recap: Sine and Cosine Functions





Absolute Positional Embeddings

- Let t be a desired position. Then the i-th element of the positional vector is:

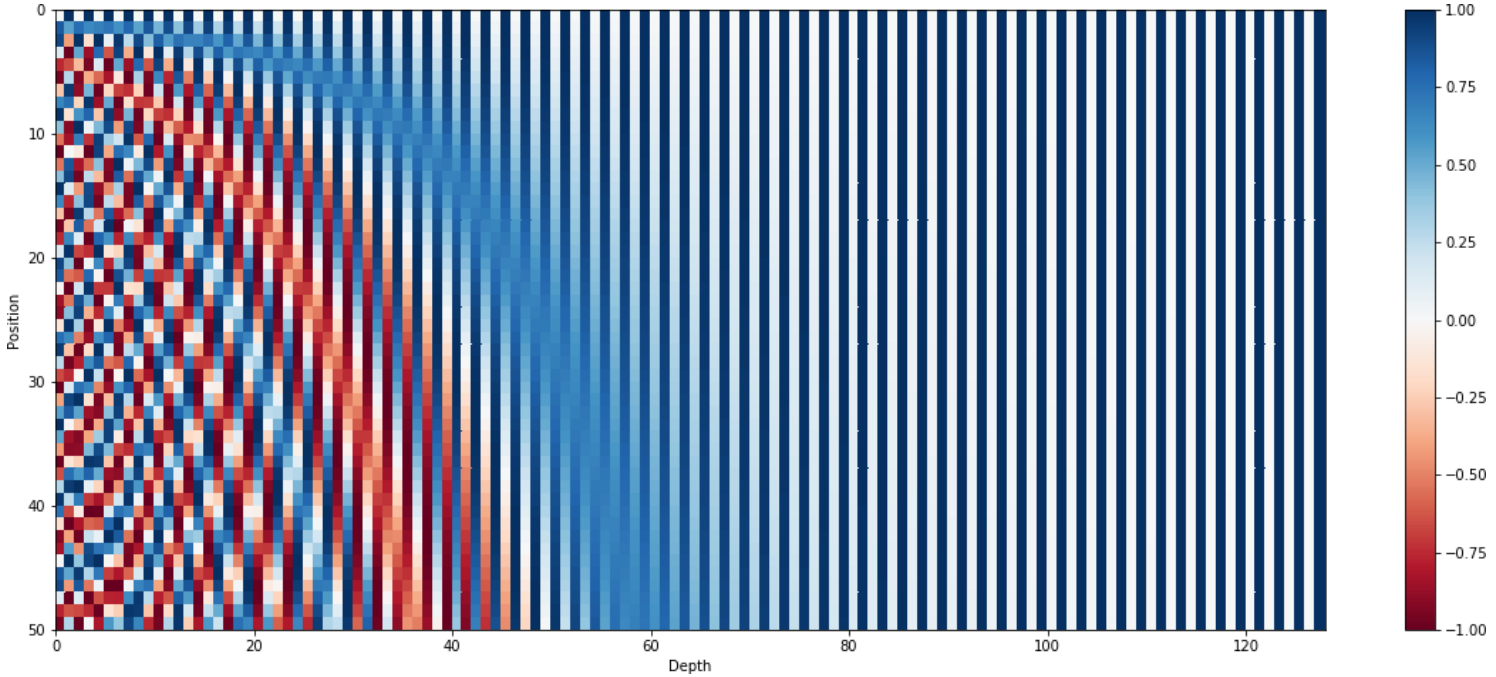
$$\overrightarrow{p_t}^{(i)} = f(t)^{(i)} := \begin{cases} \sin(\omega_k . t), & \text{if } i = 2k \\ \cos(\omega_k . t), & \text{if } i = 2k+1 \end{cases} \qquad \omega_k = \frac{1}{10000^{2k/d}}$$

- Here d is the maximum dimension of each positional vector.
- This provides unique vectors for each position.

https://kazemnejad.com/blog/transformer_architecture_positional_encoding/

Visualizing Absolute Positional Embeddings

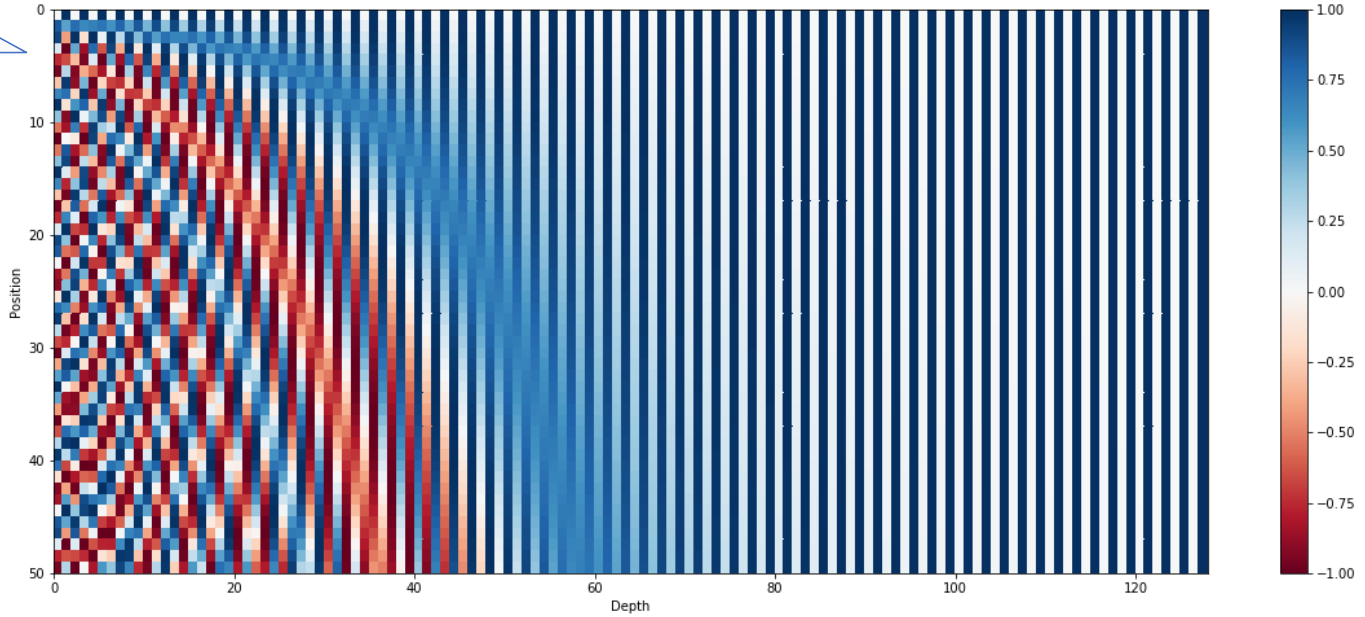
- Here positions range from 0-50, for an embedding dimension of 130.



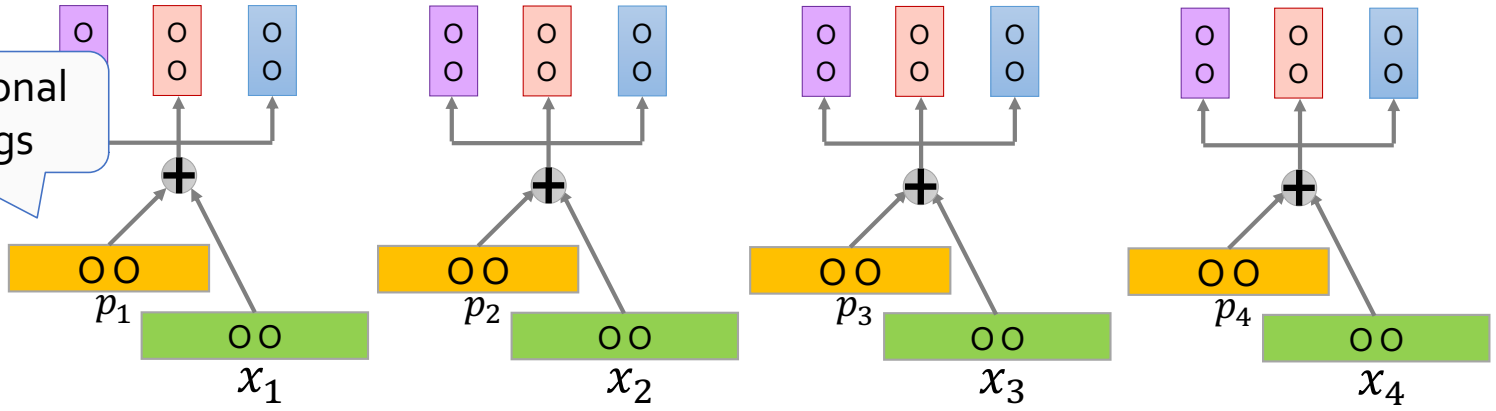
Notice, **any pair of rows are different!**

Sine/Cosine encoding

$$p_i = \begin{pmatrix} \sin(i/10000^{2*1/d}) \\ \cos(i/10000^{2*1/d}) \\ \vdots \\ \sin(i/10000^{2*\frac{d}{2}/d}) \\ \cos(i/10000^{2*\frac{d}{2}/d}) \end{pmatrix}$$



p_i are positional embeddings



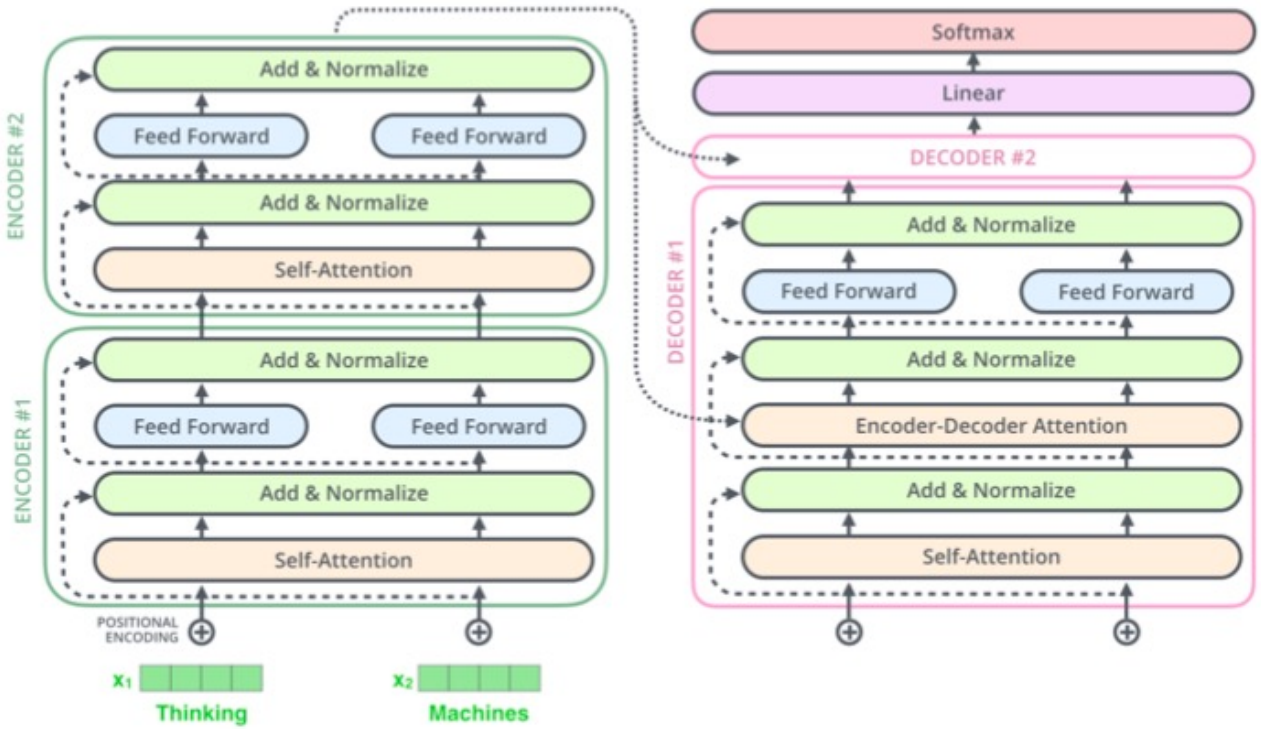
Recap— positional encoding

- They inform Transformer about word orderings.

- The original proposal: fixed sine/cosine embeddings added to word embeddings.
  - While this is a reasonable start, later we will see more sophisticated alternatives.

- Next: let's think about running transformer on language data.

Tranfsormer Architecture



1) https://jalammar.github.io/illustrated-transformer/
2) http://nlp.seas.harvard.edu/annotated-transformer/

Outline

- Transformer,
- **Pretraining LLMs**
- LLMs Instruction
- Prompting LLMs
- Parameter Efficient Fine Tuning

The nature of token

- Thisisatextthathasnospacecharactersalmostunreadable!

The nature of token

- Thisisatextthathasnospacecharactersalmostunreadable!
- This is a text that has no space characters almost unreadable !
- This: token. isht: not a token
- Token: a string frequent enough in the data..

The most frequent tokens [1]

Token	Frequency	Percentage
the	23135851	5.14%
of	13151927	2.92%
and	12997679	2.89%
to	12136980	2.70%
a	9081179	2.02%
in	8469406	1.88%
for	5933322	1.32%
is	4705748	1.05%
on	3750421	0.83%
that	3743156	0.83%

Some Least frequent tokens [2]

Token	Freq.	Meaning
xylitol	10	A sugar alcohol used as a sweetener and a dental health product
zydeco	10	A kind of black American dance music originally from Louisiana, ..
abalone	11	A large edible sea snail with flattened shell lined with mother-of-pearl
baccarat	11.	A card game in which players hold two or three card hands…
cabernet	11	A type of red wine grape or wine made from it
dabbling	11	Taking part in an activity in a casual or superficial way
eardrum	11	A thin membrane that forms part of the middle ear, .,,
fabulist	11	A person who composes or relates fables, or a liar
gabbro	11	A dark coarse-grained igneous rock consisting of plagioclase …

[1] https://stackoverflow.com/questions/35086440/python-how-to-compute-the-top-x-most-frequently-used-words-in-an-nltk-corpus
[2] https://unscramblex.com/anagram/various

Tokenization

- Out-of-vocabulary, decomposition, Sinohydro ⇒ Sino, hydro

counts	
5	l o w _
2	l o w e s t _
6	n e w e r _
3	w i d e r _
2	n e w _

merge	current vocabulary
	_, d, e, i, l, n, o, r, s, t, w
(e, r)	_, d, e, i, l, n, o, r, s, t, w, er
(er, _)	_, d, e, i, l, n, o, r, s, t, w, er, er_
(n, e)	_, d, e, i, l, n, o, r, s, t, w, er, er_ , ne
(ne, w)	_, d, e, i, l, n, o, r, s, t, w, er, er_ , ne, new

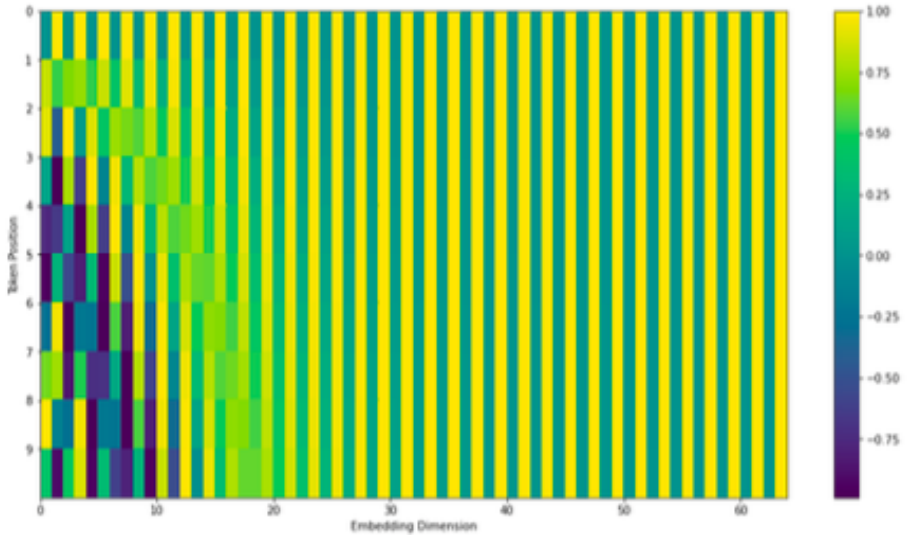
Table: Corpus, merges, and vocabulary.[1] _ denotes end-of-word symbol.

- other subword tokenization algorithms: WordPiece, SentencePiece, Unigram.

1) https://web.stanford.edu/~jurafsky/slp3/

Positional Encoding

$$PE(pos, 2i) = sin(pos/10000^{2i/d_{model}})$$
$$PE(pos, 2i+1) = cos(pos/10000^{2i/d_{model}})$$



pos encoding + token encoding
i: position in the embedding vector

https://machinelearningmastery.com/a-gentle-introduction-to-positional-encoding-in-transformer-models-part-1/

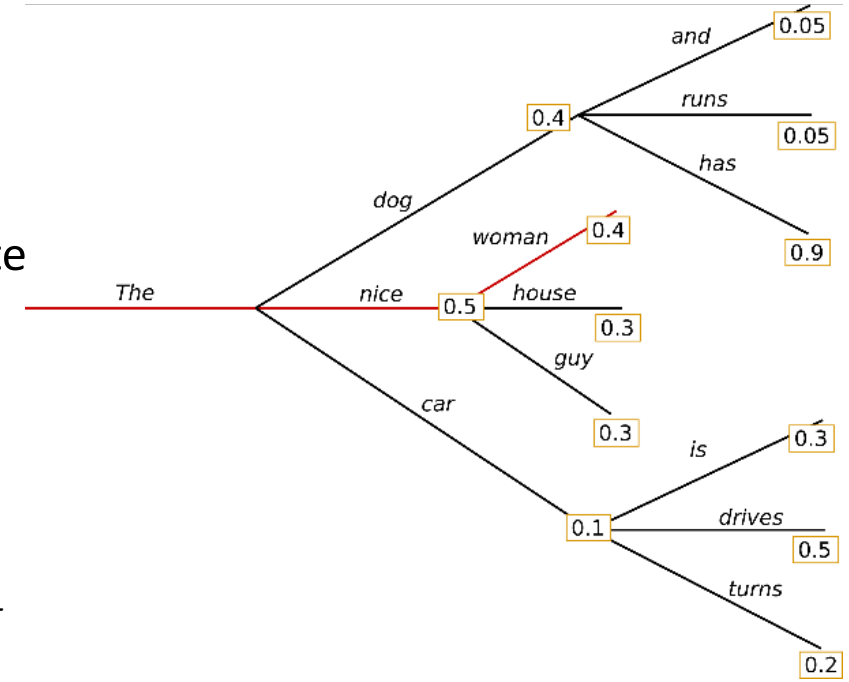
Decoding – predicting the next token…

Greedy search: $w_t = \text{argmax}_w P(w \mid w_{1:t-1}) \Rightarrow$ *"The nice woman"*

Beam search: keep the most likely *num_beams* of hypotheses at each time ste ⇒ *"The dog has"*

Sampling: $w_t \sim P(w \mid w_{1:t-1})$

- $\frac{exp(y_j/T)}{\sum_{k-1}^{n} exp(y_k/T)}$ *T*: temperature.
  - low **temperature** softmax: [0.01,0.01,0.98] ⇒ *controlled,*
  - high temperature softmax: [0.2,0.2,0.6] ⇒ *open-ended*
- **Top-K**: choose top-K tokens with highest probabilities, remove the rest, adjust their probabilities to sum to 1.
- **Top-p** (nucleus) sampling: choose from the smallest possible set of words whose cumulative probability exceeds the probability p.



https://huggingface.co/blog/how-to-generate

Outline

- Transformer
- **Pretraining LLMs**
- LLMs Instruction
- Prompting LLMs
- Parameter Efficient Fine Tuning

Pretraining Large Language Models

- Pre-training objectives:
  - Masked LM, Causal LM, PrefixLM, denoising
- General purpose: modelling language + knowledge
- Encoder Only (Bert)
- Encoder Decoder (T5)
- Decoder Only (GPT)

Language Model in LLMs…

Models that assign probabilities to sequences of words are called **language models**

- *P*(all of a sudden I notice three guys standing on the sidewalk) > *P*(on guys all I of notice sidewalk three a sudden standing the)
- *P*(?|Please turn your homework ?) ⇒ in > over > > the > elephant *uni-directional*
  - shorter context size, less accurate prediction: *P*(?|your homework ?)
- *P*(?|Please turn ? homework) *bi-directional*
- *P*(he briefed reporters on the main contents of the statement | il a informé les journalistes des principaux éléments de la déclaration) >

  P(he to reporters introduced main content | il a informé les journalistes des principaux éléments de la déclaration)

  sequence-to-sequence

1) https://web.stanford.edu/~jurafsky/slp3/

BERT |Decoder Only | MLM

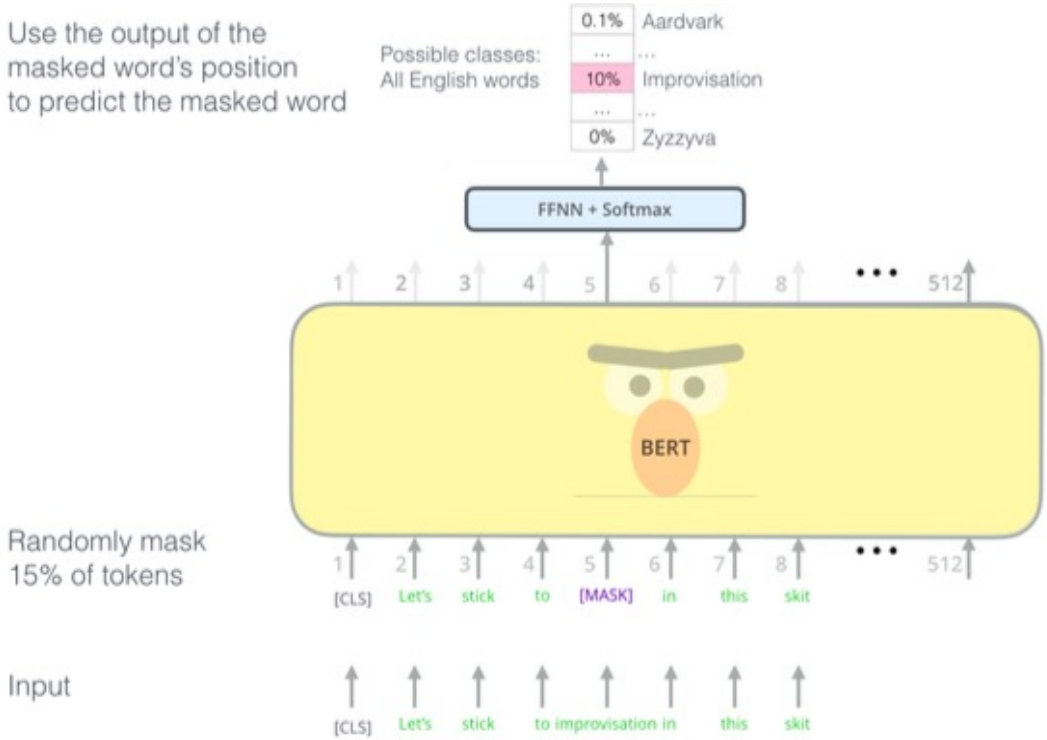
- Masked Language Modeling (MLM) predicts a masked token in a sequence, and the model can attend to tokens bidirectionally [1].
- `*Distributional hypothesis* of language?
  - Word in a context are semantically similar

- Contextualized word embeddings:

  Apple presented the new iphone X

  vs

  She picked a ripe apple from the tree



[1] http://jalammar.github.io/illustrated-bert/

Sentence Bert

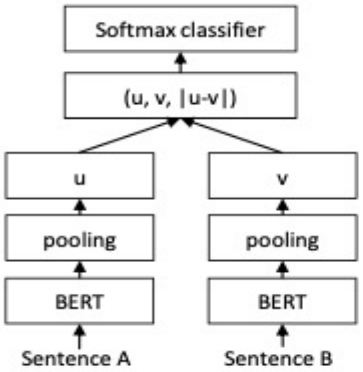


Figure: SBERT architecture with classification objective function, e.g., for fine-tuning on SNLI dataset. The two BERT networks have tied weights (siamese network structure).

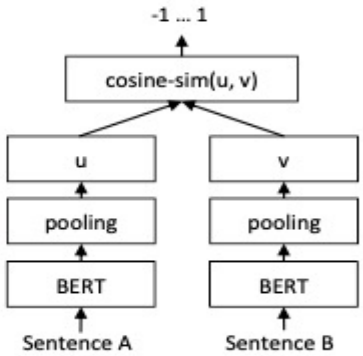


Figure: SBERT architecture at inference, for example, to compute similarity scores. This architecture is also used with the regression objective function.

- Semantic textual similarity: `https://www.sbert.net`

```
model = SentenceTransformer("all-MiniLM-L6-v2")
emb1 = model.encode("This is a red cat with a hat.")
emb2 = model.encode("Have you seen my red cat?")
cos_sim = util.cos_sim(emb1, emb2)
```

- Keyword extraction: `https://github.com/MaartenGr/KeyBERT`

1) Reimers, Nils, and Iryna Gurevych. "Sentence-bert: Sentence embeddings using siamese bert-networks." arXiv preprint arXiv:1908.10084 (2019).

Improving Language Understanding by Generative Pre-Training (GPT)

GPT – Unsupervised pretraining

- Data:
  - BooksCorpus dataset:
    - over 7,000 unique unpublished books from a variety of genres including Adventure, Fantasy, and Romance.
    - contains long stretches of contiguous text, allows the generative model to learn to condition on long-range information.
  - 1B Word Benchmark used by ELMo - approximately the same size

Improving language understanding by generative pre-training, A Radford, K Narasimhan, T Salimans, I Sutskever, https://www.mikecaptain.com/resources/pdf/GPT-1.pdf

GPT, Decoder-only, CLM

Causal language modeling (CLM) predicts the next token in a sequence of tokens, and the model can only attend to tokens on the left.

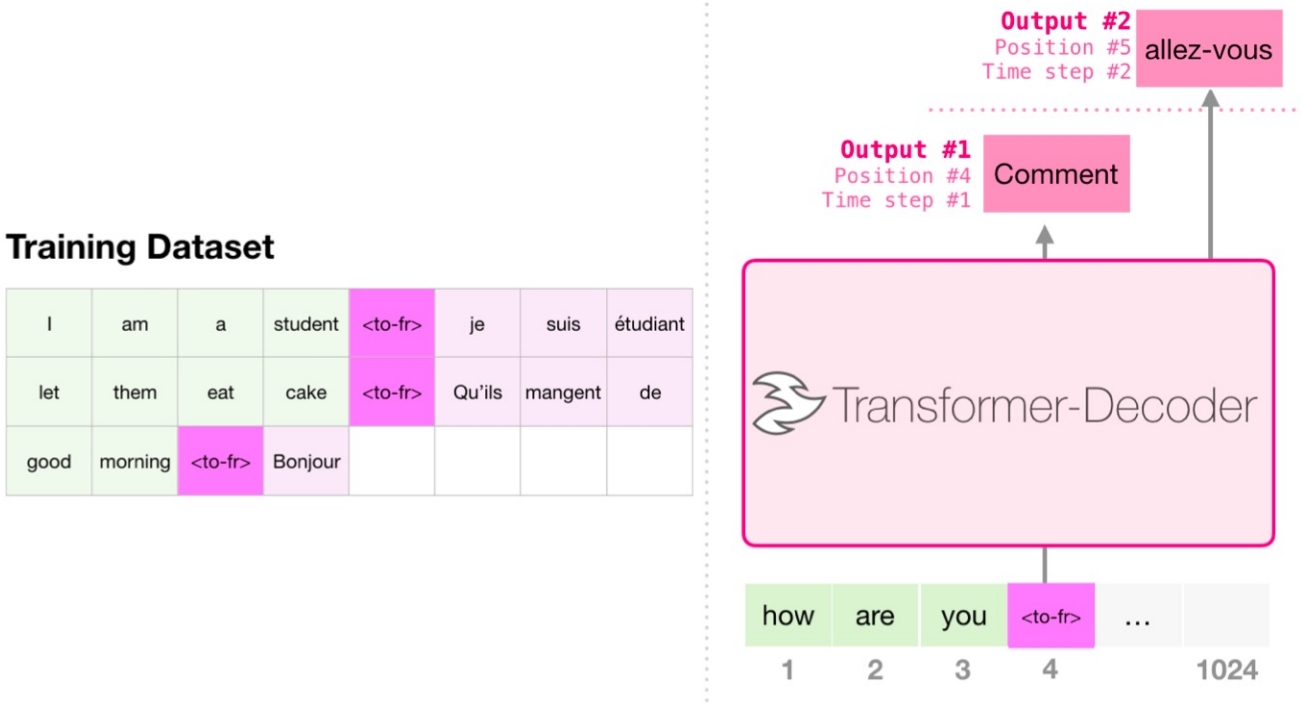


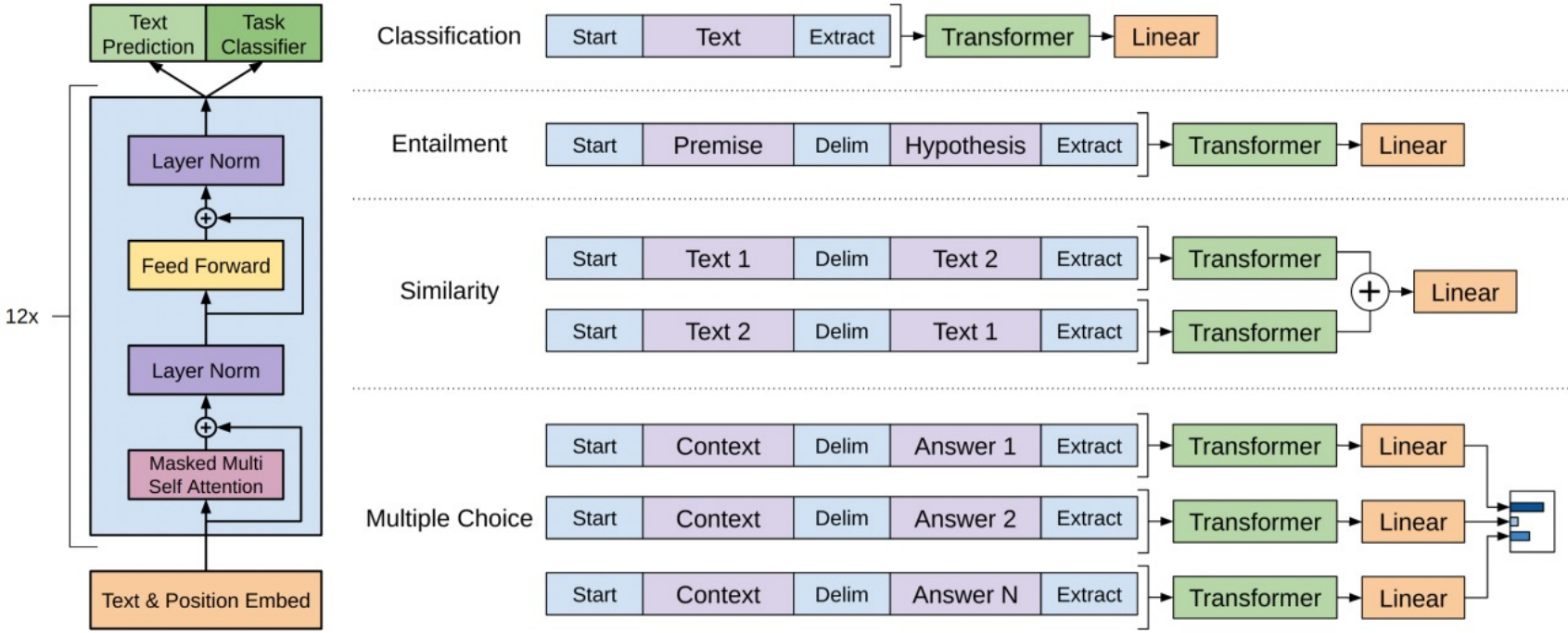
Figure: GPT: **outputs one token at a time.**[1] <to-fr> can be generalised into natural language instructions.

[1] https://jalammar.github.io/illustrated-gpt2/

GPT – model specifications

- largely follows the original transformer work
- *12-layer decoder-only transformer* with masked self-attention heads (768 dimensional states and 12 attention heads).
- train for 100 epochs on minibatches of 64 randomly sampled, contiguous sequences of 512 tokens.
- Regularization: attention dropouts with a rate of 0.1
- position embeddings learned instead of the sinusoidal proposed in original work.

GPT – Supervised fine tuning



GPT – Supervised finetuning

- Natural Language Inference (entailement)

Method	MNLI-m	MNLI-mm	SNLI	SciTail	QNLI	RTE
ESIM + ELMo [44] (5x)	-	-	89.3	-	-	-
CAFE [58] (5x)	80.2	79.0	89.3	-	-	-
Stochastic Answer Network [35] (3x)	80.6	80.1	-	-	-	-
CAFE [58]	78.7	77.9	88.5	83.3		
GenSen [64]	71.4	71.3	-	-	82.3	59.2
Multi-task BiLSTM + Attn [64]	72.2	72.1	-	-	82.1	**61.7**
Finetuned Transformer LM (ours)	**82.1**	**81.4**	**89.9**	**88.3**	**88.1**	56.0

- Q&A

Method	Story Cloze	RACE-m	RACE-h	RACE
val-LS-skip [55]	76.5	-	-	-
Hidden Coherence Model [7]	77.6	-	-	-
Dynamic Fusion Net [67] (9x)	-	55.6	49.4	51.2
BiAttention MRU [59] (9x)	-	60.2	50.3	53.3
Finetuned Transformer LM (ours)	**86.5**	**62.9**	**57.4**	**59.0**

- Semantic similarity and classification

Method	**Classification**		**Semantic Similarity**			**GLUE**
	CoLA (mc)	SST2 (acc)	MRPC (F1)	STSB (pc)	QQP (F1)	
Sparse byte mLSTM [16]	-	**93.2**	-	-	-	-
TF-KLD [23]	-	-	**86.0**	-	-	-
ECNU (mixed ensemble) [60]	-	-	-	81.0	-	-
Single-task BiLSTM + ELMo + Attn [64]	35.0	90.2	80.2	55.5	66.1	64.8
Multi-task BiLSTM + ELMo + Attn [64]	18.9	91.6	83.5	72.8	63.3	68.9
Finetuned Transformer LM (ours)	**45.4**	91.3	82.3	**82.0**	**70.3**	**72.8**

T5, Encoder-decoder, MLM

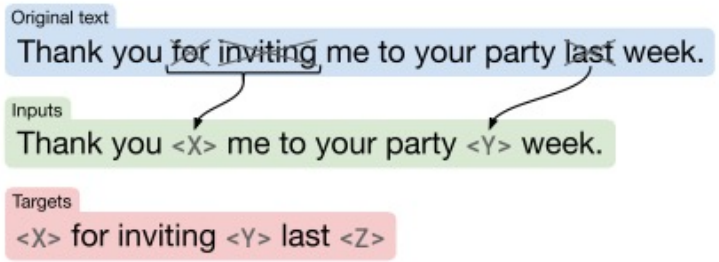


Figure: **PRE-TRAINING**: The words "for", "inviting" and "last" (marked with an ×) are randomly chosen for corruption. Each consecutive span of corrupted tokens is replaced by a sentinel token (shown as <X> and <Y>) that is unique over the example. The output sequence then consists of the dropped-out spans, delimited by the sentinel tokens used to replace them in the input plus a final sentinel token <Z>.

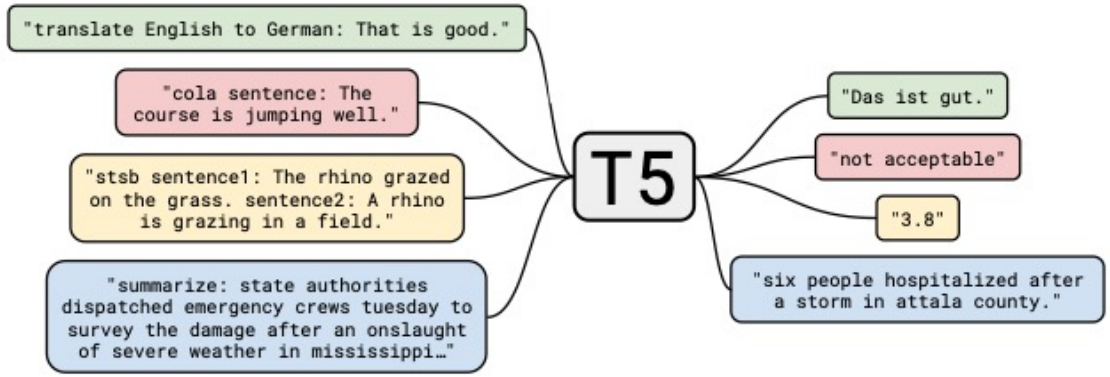
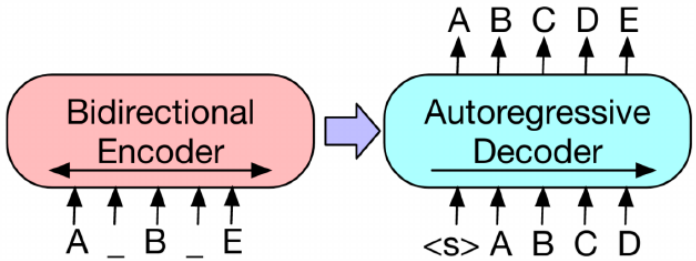


Figure: **INSTRUCTION-TUNING**: every task is cast as feeding text as input to generate some target text.[1]

1) Raffel, Colin, et al. "Exploring the limits of transfer learning with a unified text-to-text transformer." The Journal of Machine Learning Research 21.1 (2020): 5485-5551.

Bart Overview

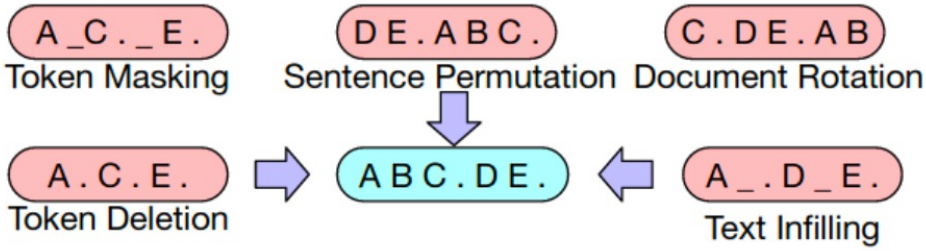


- A denoising autoencoder, that uses a bidirectional encoder and a left-to-right autoregressive decoder.
- Uses the standard Transformer architecture, with 12 encoder layers and 12 decoder layers.
- Pre-trained on 160GB of english text.
- Given its architecture, Bart can be used in both discriminative and generative tasks.

BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension

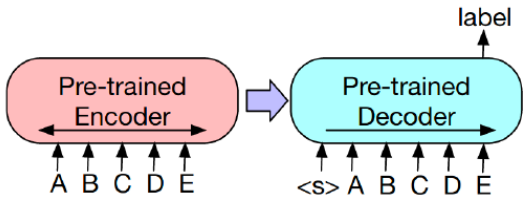
Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Ves Stoyanov, Luke Zettlemoyer

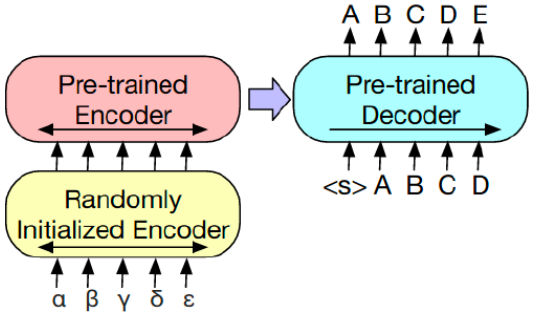
Bart – Pre-training Task



- The Pre-training task is to denoise a corrupted text. Several noising functions are tested:
  - Token Masking: random tokens are replaced with a [MASK] token.
  - Token Deletion: random tokens are deleted.
  - Text Infilling: Random text spans are sampled with span length drawn from Poisson distribution ($\lambda = 3$) and replaced with [MASK] token.
  - Sentence Permutation: Sentences are shuffled.
  - Sentence Rotation: sentences are rotated.
- Ablation experiments show that a combination of text infilling and sentence permutation perform consistently good on supervised tasks.

Bart – Fine tuning





- Bart can be fine-tuned on different types of tasks:
  - **Sequence Generation:** Given the decoder is autoregressive, Bart can be directly used to generate sequences in the same language as the input sequence.
  - **Sequence classification:** The final hidden state of the final decoder token is used. (Figure to the left)
  - **Tokens classification:** The top hidden state of the each decoder token is used.
  - **Machine translation:** The encoder embedding layer is replaced with an encoder that encodes the source language. (Figure to the right)

Bart – Evaluation

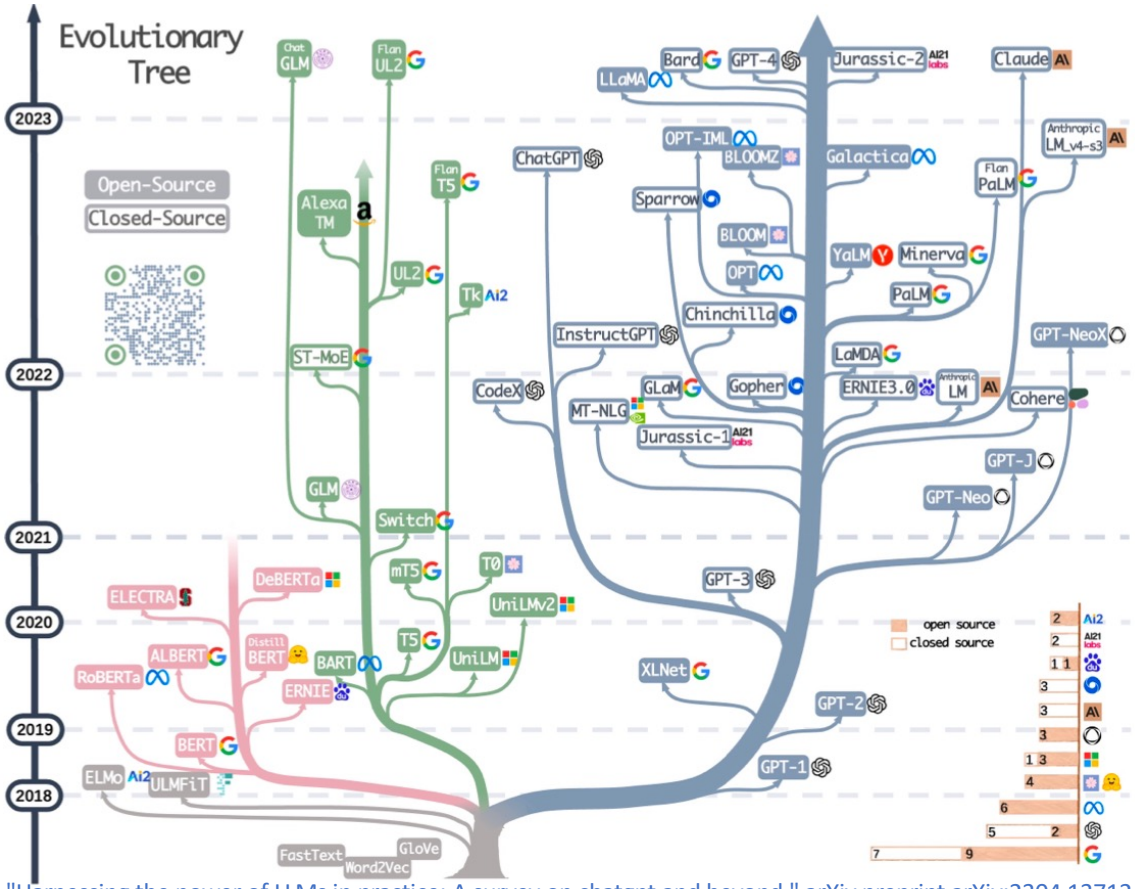
	SQuAD 1.1 EM/F1	SQuAD 2.0 EM/F1	MNLI m/mm	SST Acc	QQP Acc	QNLI Acc	STS-B Acc	RTE Acc	MRPC Acc	CoLA Mcc
BERT	84.1/90.9	79.0/81.8	86.6/-	93.2	91.3	92.3	90.0	70.4	88.0	60.6
UniLM	-/-	80.5/83.4	87.0/85.9	94.5	-	92.7	-	70.9	-	61.1
XLNet	**89.0**/94.5	86.1/88.8	89.8/-	95.6	91.8	93.9	91.8	83.8	89.2	63.6
RoBERTa	88.9/**94.6**	**86.5/89.4**	**90.2/90.2**	96.4	92.2	94.7	**92.4**	86.6	**90.9**	**68.0**
BART	88.8/**94.6**	86.1/89.2	89.9/90.1	**96.6**	**92.5**	**94.9**	91.2	**87.0**	90.4	62.8

	CNN/DailyMail			XSum		
	R1	R2	RL	R1	R2	RL
Lead-3	40.42	17.62	36.67	16.30	1.60	11.95
PTGEN (See et al., 2017)	36.44	15.66	33.42	29.70	9.21	23.24
PTGEN+COV (See et al., 2017)	39.53	17.28	36.38	28.10	8.02	21.72
UniLM	43.33	20.21	40.51	-	-	-
BERTSUMABS (Liu & Lapata, 2019)	41.72	19.39	38.76	38.76	16.33	31.15
BERTSUMEXTABS (Liu & Lapata, 2019)	42.13	19.60	39.18	38.81	16.50	31.27
BART	**44.16**	**21.28**	**40.90**	**45.14**	**22.27**	**37.25**

mBart – Multilingual Bart

- Multilingual sequence-to-sequence denoising auto-encoder.
- Follows BART architecture and objectives, but trained on several mono-language corpora across several languages.
- Can be directly fine-tuned on machine translation supervised tasks.
- Shows important performance gain in low-resource settings.
- mBART25 pre-trained model is published: trained on 1369.6 GB of raw text data from 25 different languages corpora.

Evolutionary tree of LLMs



1) Yang, Jingfeng, et al. "Harnessing the power of LLMs in practice: A survey on chatgpt and beyond." arXiv preprint arXiv:2304.13712 (2023).

Scaling laws for LLMs

Scaling up the amount of data D, compute power C, and model parameter $N \Rightarrow$ **predictable smooth continued** increase in model performance (loss L).
Such power-law relationship is called a scaling law (read more here[3]).

$$f(x) = ax^{-k} + \epsilon \tag{1}$$

where ϵ denotes a deviation from the power-law function. k is the slope.

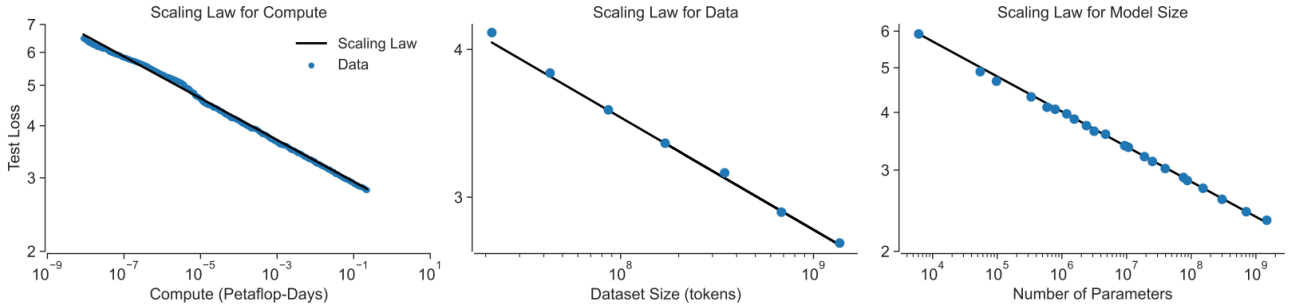


Figure: Scaling laws[1] reliably predict that model performance (y-axes) improves with increasing compute **(Left)**, training data **(Middle)**, and model size **(Right)**. In all cases a power-law (straight line, black) fits the empirically observed data (blue) exceptionally well. Figure adapted from 2. Note that x-axis and y-axis are logarithmic.

---

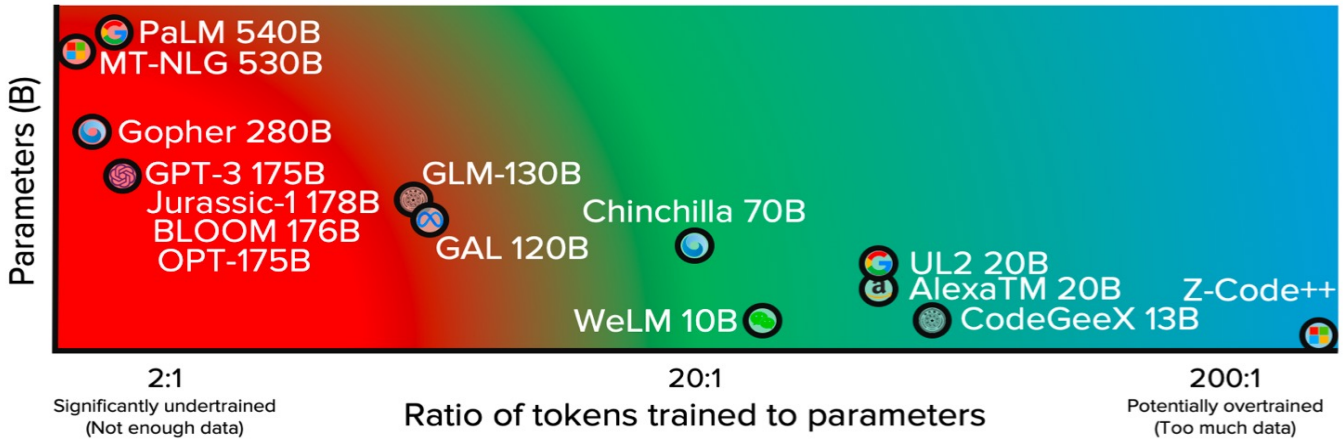
1) Ganguli, Deep, et al. "Predictability and surprise in large generative models." Proceedings of the 2022 ACM Conference on Fairness, Accountability, and Transparency. 2022.

2) Kaplan, Jared, et al. "Scaling laws for neural language models." arXiv preprint arXiv:2001.08361 (2020).

3) `https://en.wikipedia.org/wiki/Neural_scaling_law`

Scaling laws for LLMs

Chinchilla (scaling) law[1,2] suggests we need 20 text tokens per parameter, i.e., 1.4T tokens for training a 70B model.



Is scaling of $D \uparrow$, $C \uparrow$, $N \uparrow$ unlimited, as it's predictable?
By one estimate[2], the world's total stock of usable text data is between 4.6T and 17.2T tokens.

---

1) `https://lifearchitect.ai/chinchilla/`

2) `https://www.lesswrong.com/posts/6Fpvch8RR29qLEWNH/chinchilla-s-wild-implications`

2) Villalobos, Pablo, et al. "Will we run out of data? An analysis of the limits of scaling datasets in Machine Learning." arXiv preprint arXiv:2211.04325 (2022).

Emergent abilities of LLMs

Abrupt jumps in certain known capabilities when increasing model size, same for unknown capabilities which are appeared to be unpredictably **emergent**[1].

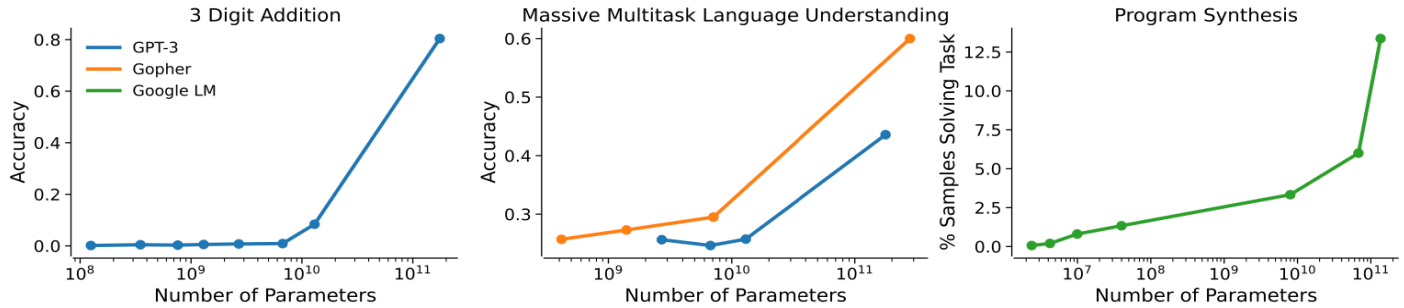


Figure: Three examples of abrupt specific capability scaling, based on three different models: GPT-3 (blue), Gopher (orange), and a Google language model (green). **(Left)** 3-Digit addition with GPT-3. **(Middle)** Language understanding with GPT-3 and Gopher. **(Right)** Program synthesis with Google language models.

Three digit addition is performed accurately less than 1% of the time on any model with less than 6B parameters, but this jumps to 8% accuracy on a 13B parameter model and 80% accuracy on a 175B parameter model.

AI safety: societally impactful potential harms (e.g., toxicity) of the model also scale.

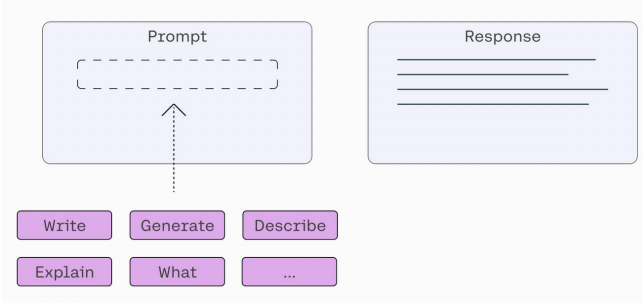
---

1) Wei, Jason, et al. "Emergent abilities of large language models." arXiv preprint arXiv:2206.07682 (2022).

Outline

- Pretraining LLMs
- **LLMs Instruction**
- Prompting LLMs
- Parameter Efficient Fine Tuning

LLMs & Instruction



InstructGPT[1] ⇒ ChatGPT

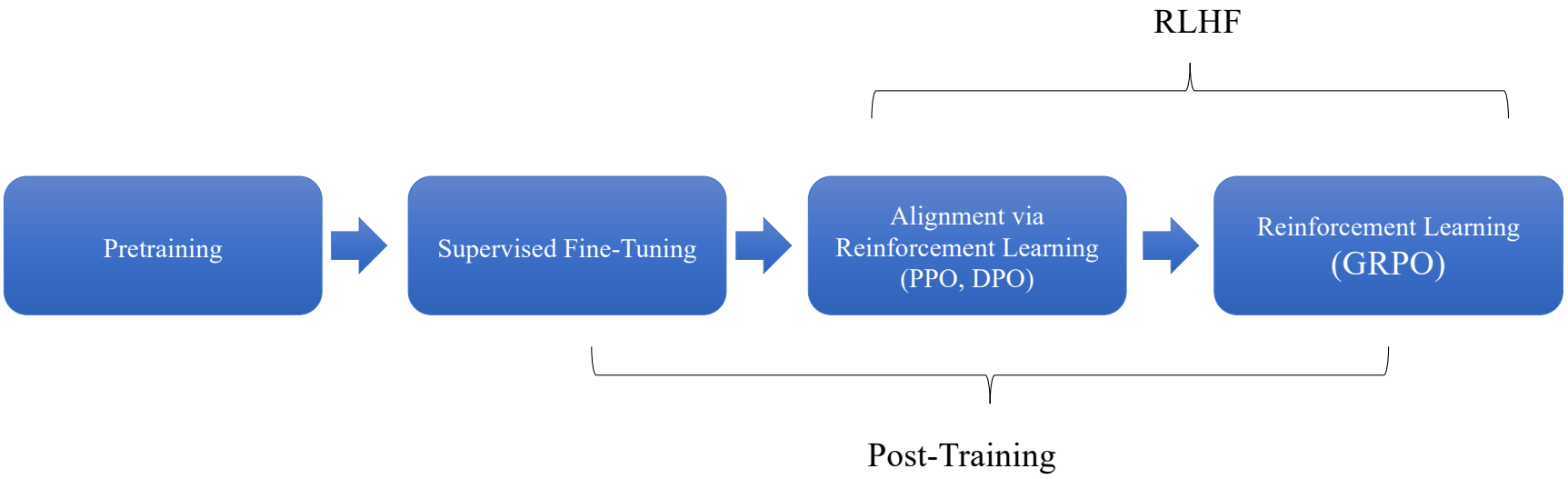
- There is as much manual labour as there is intelligence.
- InstructGPT = GPT (pre-training) + Instruction-tuning (multi-task fine-tuning) + Alignment-tuning (RLHF: Reinforcement Learning from Human Feedback)
- GPT trained for predicting the next token ⇒ However, what we want is "*To follow the user's instructions* (*instruction-tuning*), be helpful, honest, and harmless."
- Making language models bigger does not inherently make them better at following a user's intent: language modelling objective (CLM) is **misaligned**.

  Solution: Fine-tuning to learn (to be aligned with) human feedbacks/preferences ⇒ Alignment-tuning (RLHF)

Human evaluation
- outputs from the 1.3B InstructGPT model are preferred to outputs from the 175B GPT-3, despite having 100x fewer parameters.

1) Ouyang, Long, et al. "Training language models to follow instructions with human feedback." Advances in Neural Information Processing Systems 35 (2022): 27730-27744.

LLM Development Pipeline



Issues of each stage

Pretraining

- *“The model reads the whole internet but doesn’t know how to help you.”*
- *Use case: Language modeling*
- *Issue: e.g. Cannot be used as a chat model.*

Supervised Fine-Tuning (SFT)

- *“We give it examples of what a good answer looks like.”*
- Use case: Summarization, Reformulation, Code, etc
- Issue: Can generate harmful content

Issues of each stage

RLHF:

Alignment

- *"We adjust it so it listens, behaves, and solves tasks better."*
- RL can be used for alignment. **DPO is not RL!**
- Use cases: Safety and behaviour alignement
- Issue: Aligned model cannot really "reason". Untill here, the model just learn to reproduce.

GRPO, DAPO, RLOO:

- RL improves the model performances on "reasoning" tasks.
- Use cases: Maths, Code, "Reasoning"
- Issue: Training is resource intensive

Instruction Tuning - Training Steps

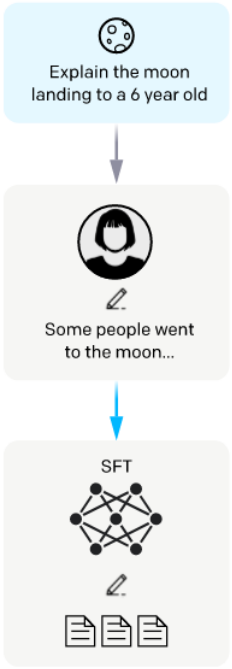
Step 1

Collect demonstration data, and train a supervised policy.

A prompt is sampled from our prompt dataset.

A labeler demonstrates the desired output behavior.

This data is used to fine-tune GPT-3 with supervised learning.



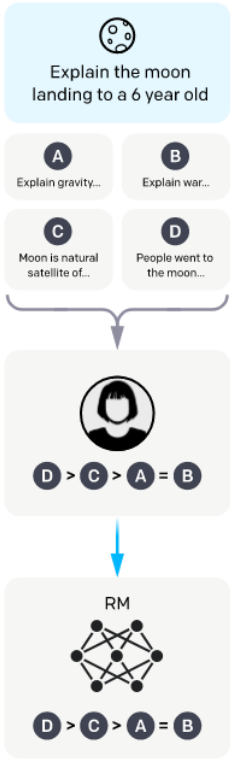
Step 2

Collect comparison data, and train a reward model.

A prompt and several model outputs are sampled.

A labeler ranks the outputs from best to worst.

This data is used to train our reward model.



Step 3

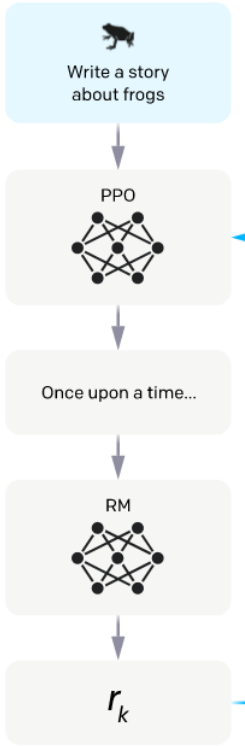
Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.

The policy generates an output.

The reward model calculates a reward for the output.

The reward is used to update the policy using PPO.



Step 1: Instruction-tuning GPT-3 ⇒ SFT model | 13k data

- **Plain**: labelers come up with an arbitrary task, while ensuring the tasks had sufficient diversity.
- **Few-shot**: labelers to come up with an instruction, and multiple query/response pairs for that instruction.
- **User-based**: use-cases stated. Labelers to come up with prompts corresponding to these use cases.

Table: Distribution of use case categories from our API prompt dataset.

Use-case	(%)
Generation	45.6%
Open QA	12.4%
Brainstorming	11.2%
Chat	8.4%
Rewrite	6.6%
Summarization	4.2%
Classification	3.5%
Other	3.5%
Closed QA	2.6%
Extract	1.9%

Table: Illustrative prompts from our API prompt dataset. These are fictional examples inspired by real usage.

Use-case	Prompt
Brainstorming	List five ideas for how to regain enthusiasm for my career
Generation	Write a short story where a bear goes to the beach, makes friends with a seal, and then returns home.
Rewrite	This is the summary of a Broadway play: """ {summary} """ This is the outline of the commercial for that play: """

team of about 40 contractors. Inter-annotator agreement: 72.6 ± 1.5%

Step 2: training a Reward Model (RM) | 33k data

RM $r_\theta(x,y)$ outputs a scalar reward indicating human preference, for a given pair of prompt x and response y, with parameters θ.

TRAINING:
For a given prompt x, present labelers with responses $y_{k=1...K\ (K = 4\ ...9)}$ to **rank** (*human preferences are expressed as comparisons*), e.g., $D > C > A = B$.

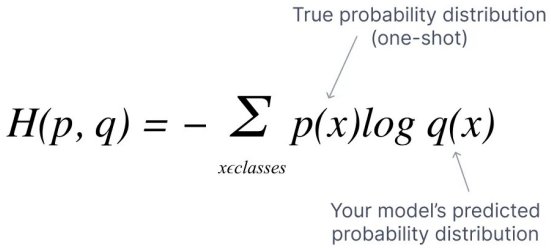
The loss function for the RM is:

$$\text{loss}(\theta) = -\frac{1}{\binom{K}{2}} E_{(x,y_w,y_l)\sim D}\left[\log\left(\sigma\left(r_\theta\left(x,y_w\right) - r_\theta\left(x,y_l\right)\right)\right)\right]$$

where y_w is the preferred response out of the pair of y_w and y_l , and D is the dataset of human comparisons.

Pretraining and SFT

- Cross Entropy Loss: Maximize probability of correct token x.



e.g.: What is the most probable word after: The cat felt on

Direct Preference Optimization (DPO)

Alignment

- Maximize probability of winner token and minimize probability of loser.

$$\mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l)\sim\mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)} \right) \right]$$

e.g.: Give me a recipe of Gratin Dauphinois

The loss function is minimized during training. Minimizing the negative log-probability (log) of a preference means **maximizing the probability** that the policy satisfies the preference.

Expectation Term

•$E(x,y_w,y_l) \sim D[\ldots]$: **expected value** (average) over all preference triplets (x,y_w,y_l) sampled from the preference dataset D.

- **x**: The **prompt** or context.
- $\mathbf{y_w}$: The **preferred** (winning) response.
- $\mathbf{y_l}$: The **dispreferred** (losing) response.

Direct Preference Optimization (DPO)

Alignment

- Maximize probability of winner token and minimize probability of loser.

$$\mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l)\sim\mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)} \right) \right]$$

e.g.: Give me a recipe of Gratin Dauphinois

- loss function is minimized during training.
- Minimizing the negative log-probability (log) of a preference means **maximizing the probability** that the policy satisfies the preference.

1. Expectation Term

•E$_{(x,y_w,y_l)\sim D}$[…]: **expected value** (average) over all preference triplets (x,y_w,y_l) sampled from the preference dataset D.

- **x**: The **prompt** or context.
- **y_w**: The **preferred** (winning) response.
- **y_l**: The **dispreferred** (losing) response.

$\pi_\theta(y|x)$: probability of generating response y given prompt x under the **current policy being trained (π_θ)**.

$\pi_{ref}(y|x)$: probability of generating response y given prompt x under the **reference policy (π_{ref})** (often the initial SFT model).

log $\pi_\theta(y|x)/\pi_{ref}(y|x)$: **log-ratio** of probability of response y under the new policy vs. the reference policy. effectively the **"reward"** or change in log-probability of y *relative* to the starting model.

β: A **temperature or scaling parameter** (a hyperparameter, typically positive): controls **strength** of penalty for deviating from reference policy and **magnitude** of optimization update. larger β means policy is more constrained to stay close to the reference policy, similar to the KL-divergence penalty in RLHF.

Direct Preference Optimization (DPO)

$$\mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l)\sim\mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)} \right) \right]$$

- difference in the *relative reward* (or log-probability ratio) for the winning response y_w and the losing response y_l.

•**Goal:** For the new policy π_θ to be effective, this entire **Preference Score** must be **positive and large;** hence π_θ must increase the log-probability of y_w relative to y_l, compared to $\pi_{ref.}$

- Sigmoid Function (σ) squashes **Preference Score** into a probability that y_w is preferred over y_l.

Final Loss Calculation

• **$\log_\sigma$(Preference Score)**: large and positive, we want the output of the sigmoid function, σ(Preference Score), to be close to 1. The **log of a number close to 1 is close to 0**.

• **–E[…]**: **negative sign** turns the maximization problem (maximizing the probability of preference) into a **minimization problem** (minimizing the loss LDPO), standard for gradient-based optimization.

Outline

- Transformer
- Pretraining LLMs
- LLMs Instruction
- **Prompting LLMs**
- Parameter Efficient Fine Tuning
-

Prompt Engineering

- Prompt engineering - ways to communicate with the models.
- Better prompting ⇒ better results.
- Zero-shot, Few-shot, Chain of Thought (CoT)

1) https://cameronrwolfe.substack.com/p/advanced-prompt-engineering

Zero Shot prompting – only the instruction

1. **Simple prompt** (Zero-shot)
   `In summary: ... | To summarize in plain language, ... | The main point to take from this article is that ...`
   ⇒ Different formulations of the same prompt which might sound similar to humans can lead to generations that are quite different from each other.
2. **Adding task description**

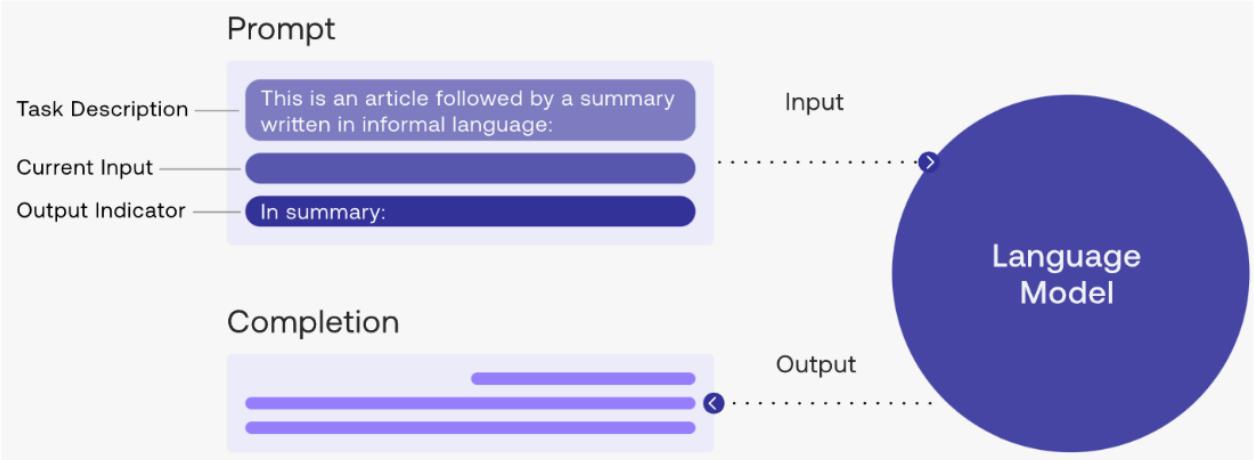


Figure: adding task description[1].

https://docs.cohere.com/docs/prompt-engineering

Few-shot prompting

Adding examples (**Few-shot** / in-context learning)

```
This is a movie review sentiment classifier.
Review: "I loved this movie!" This review is positive.
Review: "I don't know, it was ok I guess.." This review is neutral.
Review: "What a waste of time, would not recommend this movie." This review is negative.
Review: "I really enjoyed this movie!" This review is ...
```

Chain-of-Thought (**Few-shot-CoT**)

- a series of intermediate natural language reasoning steps that lead to the final output ⇒ improves complex reasoning capability of LLMs.
- *CoT prompting is an emergent ability of model scale*, it only yields performance gains when used with models of sufficient scale (100B+ parameters).

Few-shot Chain-of-Thought

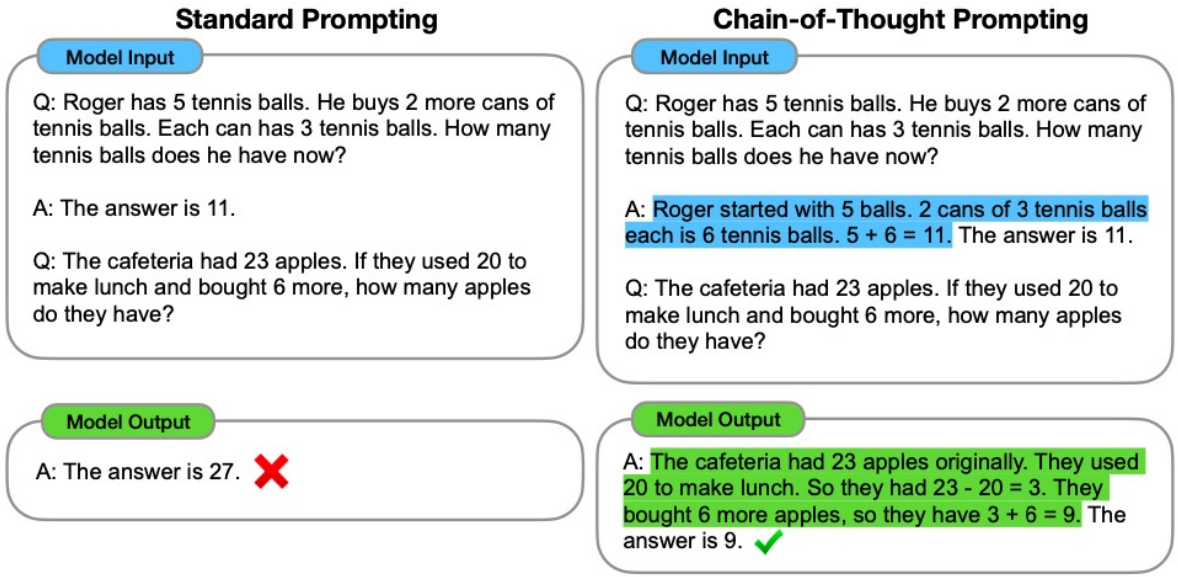


Figure: Chain-of-thought prompting enables large language models to tackle complex arithmetic, commonsense, and symbolic reasoning tasks. Chain-of-thought reasoning processes are highlighted.

Self-consistency[1]: generate multiple chains of thought and taking the majority vote of these multiple outputs as the final answer.

1) Wang, Xuezhi, et al. "Self-consistency improves chain of thought reasoning in language models." arXiv preprint arXiv:2203.11171 (2022).

Zero-shot Chain-of-Thought

Let's think step by step. (Zero-shot-CoT)[1]

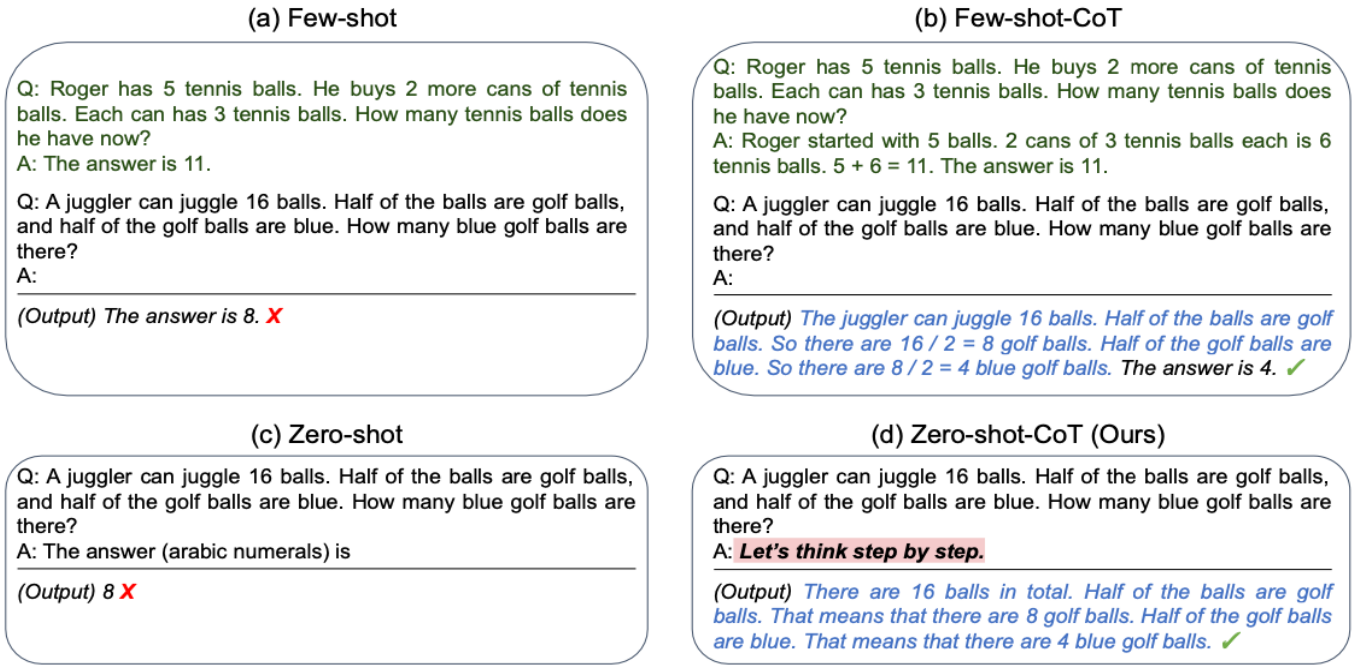


Figure: Example inputs and outputs of GPT-3 with (a) standard Few-shot, (b) Few-shot-CoT, (c) standard Zero-shot, and (d) ours (Zero-shot-CoT). Similar to Few-shot-CoT, Zero-shot-CoT facilitates multi-step reasoning (blue text) and reach correct answer where standard prompting fails. Unlike Few-shot-CoT using step-by-step reasoning examples **per task**, ours does not need any examples and just uses the same prompt "Let's think step by step" *across all tasks* (arithmetic, symbolic, commonsense, and other logical reasoning tasks).

1) Kojima, Takeshi, et al. "Large language models are zero-shot reasoners." Advances in neural information process- ing systems 35 (2022): 22199-22213.

Prompting methods- Comparison

- Comparison with baseline methods using accuracies on MultiArith and GSM8K benchmarks.
- text-davinci-002 InstructGPT used as the model if not specified.
- used the same 8 examples for Few-shot and Few-shot-CoT settings. (*1)
- To verify the variance of changing examples, report two results for 4-shot-cot by splitting the eight examples into two groups.
- (*2) insert “Let’s think step by step.” at the beginning of answer part of each examples for Few-shot-CoT to test performance gains.
- LLMs decent zero-shot reasoners by simply adding "Let's think step by step" before each answer.
- Zero-shot-CoT, using the same single prompt template, significantly outperforms zero-shot LLM performances on diverse benchmark reasoning tasks including arithmetics

MultiArith

Example 1

Gwen was organizing her book case making sure each of the shelves had exactly 9 books on it. She has 2 types of books - mystery books and picture books. If she had 3 shelves of mystery books and 5 shelves of picture books, how many books did she have in total?

GSM8K

Problem: Beth bakes 4, 2 dozen batches of cookies in a week. If these cookies are shared amongst 16 people equally, how many cookies does each person consume?
Solution: Beth bakes 4 2 dozen batches of cookies for a total of 4*2 = <<4*2=8>>8 dozen cookies
There are 12 cookies in a dozen and she makes 8 dozen cookies for a total of 12*8 = <<12*8=96>>96 cookies
She splits the 96 cookies equally amongst 16 people so they each eat 96/16 = <<96/16=6>>6 cookies
Final Answer: 6

	MultiArith	GSM8K
Zero-Shot	**17.7**	**10.4**
Few-Shot (2 samples)	33.7	15.6
Few-Shot (8 samples)	33.8	15.6
Zero-Shot-CoT	**78.7**	**40.7**
Few-Shot-CoT (2 samples)	84.8	41.3
Few-Shot-CoT (4 samples : First) (*1)	89.2	-
Few-Shot-CoT (4 samples : Second) (*1)	90.5	-
Few-Shot-CoT (8 samples)	93.0	48.7
Zero-Plus-Few-Shot-CoT (8 samples) (*2)	**92.8**	**51.5**
Finetuned GPT-3 175B	-	33
Finetuned GPT-3 175B + verifier	-	55
PaLM 540B: Zero-Shot	**25.5**	**12.5**
PaLM 540B: Zero-Shot-CoT	**66.1**	**43.0**
PaLM 540B: Zero-Shot-CoT + self consistency	**89.0**	**70.1**
PaLM 540B: Few-Shot	-	17.9
PaLM 540B: Few-Shot-CoT	-	56.9
PaLM 540B: Few-Shot-CoT + self consistency	-	74.4

1) Kojima, Takeshi, et al. "Large language models are zero-shot reasoners." Advances in neural information processing systems 35 (2022): 22199-22213.

Outline

- Transformer
- Pretraining LLMs
- LLMs Instruction
- Prompting LLMs
- **Parameter Efficient Fine Tuning**
-

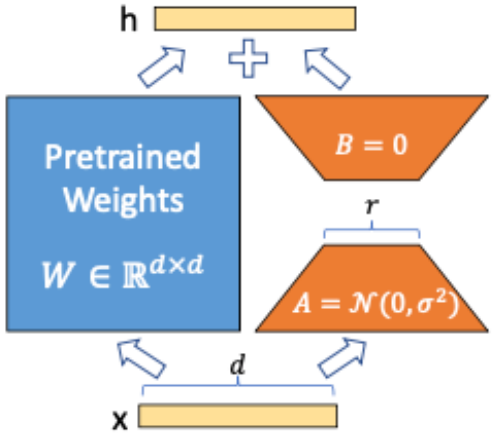
Parameter-Efficient Fine-tuning

- **Fine-tuning :** a model already trained for a given task (or general purpose) is *tuned (trained) to make it perform a second (or specific) task*.
- **Full** fine-tuning *modifies all the parameters:* storing a full copy of the model for each task is necessary
- **Parameter-Efficient** fine-tuning:
  - *freezes all or most of the pretrained parameters*
  - *augments the model with small trainable modules*
  - one line of research considers removing parameters [1][2]).
- Main methods: Adapter, Prefix, LoRA

1) Radiya-Dixit, Evani, and Xin Wang. "How fine can fine-tuning be? learning efficient language models." International Conference on Artificial Intelligence and Statistics. PMLR, 2020.
2) Zhao, Mengjie, et al. "Masking as an efficient alternative to finetuning for pretrained language models." arXiv preprint arXiv:2004.12406 (2020).

LoRA

- ***Low-Rank Adaptation***[1] freezes the pre-trained model weights *W* and injects trainable rank decomposition matrices *A*, *B* into each layer of the Transformer ⇒ *W* + *BA*.
- the learned over-parametrized models in fact reside on a **low intrinsic dimension** [2,3].
- Assume change in weights during model adaptation also has a low "intrinsic rank".

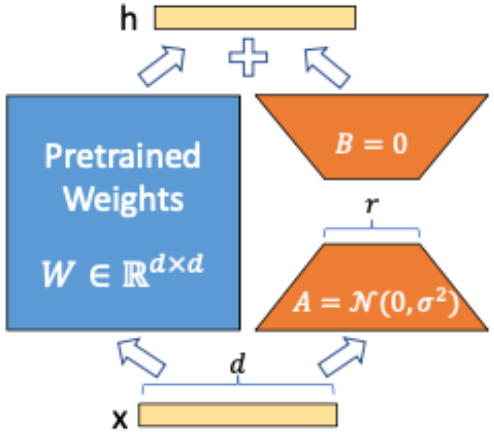


The re-parametrization. We only train *A* and *B*. ***r* can be one or two.**
Weights trained for A and B: $2dr << d^2$ *if* $r << d$

1) Hu, Edward J., et al. "Lora: Low-rank adaptation of large language models." arXiv preprint arXiv:2106.09685 (2021).
2) Li, Chunyuan, et al. "Measuring the intrinsic dimension of objective landscapes." arXiv preprint arXiv:1804.08838 (2018).
3) Aghajanyan, Armen, Luke Zettlemoyer, and Sonal Gupta. "Intrinsic dimensionality explains the effectiveness of language model fine-tuning." arXiv preprint arXiv:2012.13255 (2020).

LoRA

- For a pre-trained weight matrix $W_0 \in \mathrm{R}^{d \times k}$, constrain its update by representing the latter with a low-rank decomposition $W_0 + \Delta W = W_0 + BA$, where $B \in \mathrm{R}^{d \times r}, A \in \mathrm{R}^{r \times k}$, and the rank $r \ll \min(d,k)$.
- During training, W_0 *is frozen* and does not receive gradient updates, *A and B contain trainable parameters*.
- W_0 and $\Delta W = BA$ are multiplied with the same input - respective output vectors are summed coordinate-wise. For $h = W_0 x$, modified forward pass yields:
  - $h = W_0 x + \Delta W x = W_0 x + BAx$
- Initialization: *A* random Gaussian, *B:* zeros: $\Delta W = BA$ is zero at the beginning of training.



LoRA

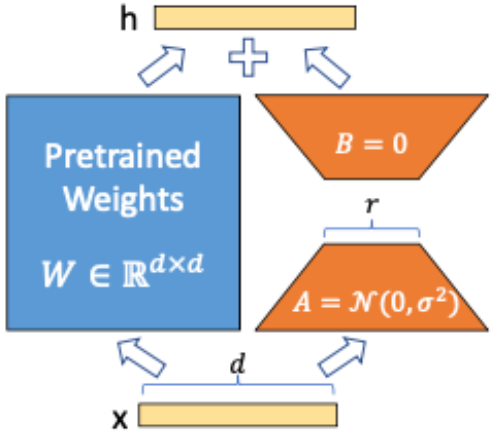
Model&Method	# Trainable Parameters	WikiSQL Acc. (%)	MNLI-m Acc. (%)	SAMSum R1/R2/RL
GPT-3 (FT)	175,255.8M	**73.8**	89.5	52.0/28.0/44.5
GPT-3 (BitFit)	14.2M	71.3	91.0	51.3/27.4/43.5
GPT-3 (PreEmbed)	3.2M	63.1	88.6	48.3/24.2/40.5
GPT-3 (PreLayer)	20.2M	70.1	89.5	50.8/27.3/43.5
GPT-3 ($Adapter^H$)	7.1M	71.9	89.8	53.0/28.9/44.8
GPT-3 ($Adapter^H$)	40.1M	73.2	**91.5**	53.2/29.0/45.1
GPT-3 (LoRA)	4.7M	73.4	**91.7**	**53.8/29.8/45.9**
GPT-3 (LoRA)	37.7M	**74.0**	**91.6**	53.4/29.2/45.1

- Performance of different adaptation methods on GPT-3 175B.
- Metrics: Validation accuracy on WikiSQL, MultiNLI-matched, and Rouge-1/2/L on SAMSum.
- LoRA performs better than prior approaches, including full fine-tuning. WikiSQL results fluctuation ±0.5%, MNLI-m ±0.1%, and SAMSum around ±0.2/±0.2/±0.1 for the three metrics.
- A more recent study[1] also confirms its effectiveness, with hyper-parameter suggestion.

1) https://www.anyscale.com/blog/fine-tuning-llms-lora-or-full-parameter-an-in-depth-analysis-with-llama-2

LoRA

- freeze the base model and efficiently switch different tasks by *replacing the matrices A and B,* reducing the storage requirement and task-switching overhead significantly.
- LoRA makes training more efficient and lowers the hardware barrier to entry by up to 3 times when using adaptive optimizers since we do not need to calculate the gradients or maintain the optimizer states for most parameters.
- only optimize the injected, much smaller low-rank matrices.
- Our simple linear design allows us to merge the trainable matrices with the frozen weights when deployed, *introducing no inference latency* compared to a fully fine-tuned model, by construction.
- LoRA is orthogonal to many prior methods and *can be combined with many of them*, such as prefix-tuning.



THANK YOU!

http://www.lix.polytechnique.fr/dascim